The CCME Release Guarantees

A formal model, with proofs, of the eight properties of §13.7c

Semen Fediukovich

September 2026

Technical report accompanying `specs/ccme-spec/SPEC.md` 2.0.0
`github.com/yohimik/dispat`

Abstract

Releasing interdependent packages is a distributed transaction without a coordinator or rollback: each publish is an irreversible write to an independent registry. CCME defines a resumable release algorithm and states eight guarantees: termination, completeness, version stability, exactly-once tag-ledger delivery, retry-invariant non-widening and convergence, channel discharge and one-way axis independence.

This report formalises the 2.0.0 algorithm and proves the guarantees under its explicit retry invariant, together with the determinism and idempotency requirements of SPEC §17.2, the duality of SPEC §13.7b, and analyses the cost and optimisation bounds of SPEC §§13.11, 20.7, qualifying them where ancestry and output work add terms. It preserves an audit of six failure classes in the 1.0.0 rules, with minimal counterexamples, repairs, and conformance vectors. CCME 2.0.0 incorporates those six repairs; the historical audit is provenance. A separate current obligation records an inconsistency between the specific graduation rule of SPEC §11.5 and its SPEC §13.9 summary.

Specification snapshot. The normative subject is released CCME 2.0.0 at repository commit `d3ce8ae637232908ced82efb5cda550f60ed2fdf`. Section 9 records one current 2.0.0 textual obligation, then separately audits CCME 1.0.0 at `713f1a0b`; the latter six defect statements and counterexamples are historical.

Contents

1 Introduction **4**

1.1 The problem 4
1.2 Why formalise them 4
1.3 Scope, and what this is not 5
1.4 Implementation examples outside the model 5
1.5 How to read it 6
1.6 Licence 7

2 The model **7**

2.1 History 7
2.2 Workspace 7

2.3 Bumps and versions . . . 8
2.4 State . . . 8
2.5 Units . . . 9
2.6 Runs . . . 10

3 Propagation **11**
3.1 The traversal . . . 11
3.2 Phase 1: the channel axis . . . 13
3.3 Phase 2: channel resolution . . . 13
3.4 Phase 3: the bump axis . . . 13

4 The publish protocol **15**
4.1 Tag after, immediately, and always . . . 15
4.2 Order . . . 15
4.3 Partial failure . . . 16

5 The eight guarantees **16**
5.1 Three lemmas . . . 17
5.2 G1: termination . . . 18
5.3 G2: completeness . . . 18
5.4 G3: version stability . . . 19
5.5 G4: no double release . . . 19
5.6 G5: no blast-radius widening . . . 20
5.7 G6: convergence . . . 20
5.8 G7: channel discharge . . . 21
5.9 G8: axis independence . . . 22

6 Determinism, idempotency, resumption **22**

7 The staleness dual **23**

8 Cost **25**
8.1 Windows . . . 26
8.2 Safe traversal reuse . . . 27
8.3 Hoisting . . . 28
8.4 Inverted channel resolution . . . 28
8.5 Parsing . . . 28

9 Proof obligations **29**
9.1 Current obligation: the graduation summary contradicts its specific rule . . . 30
9.2 Historical 1.0.0 obligations . . . 30

9.3 The prerelease window never discharges 30
9.4 Window-scoped suppression lapses, so the admitted set can grow 31
9.5 The staleness dual omits the resolvability test 33
9.6 Axis independence is one-directional 33
9.7 Inherited channels read a baseline that release moves 34
9.8 Bucketed traversal over-admits a unit's own sources 35

10 Conformance **35**

A Notation **38**

1 Introduction

1.1 The problem

A workspace is a repository holding many independently versioned packages that depend on one another. Releasing it means choosing a new version for each package that needs one, writing those versions into manifests, uploading artefacts to registries, and recording what happened.

The recording is the hard part. Registries have no transactions. A publish is an irreversible write to a service the release tool does not control, several registries may be involved (npm, a container registry, a language-specific index), and there is no coordinator that can commit or abort them as a unit. A run that publishes seven packages is seven independent, individually fallible writes; it may finish having done four of them.

CCME[1] answers this the way a saga does. There is no external state file, no lock service, and no registry polling. The release plan is a pure function of the repository at a fixed commit; each completed leg of the transaction is recorded by an annotated git tag written immediately after that leg succeeds; and recovery is re-execution: run the engine again and it recomputes the same transaction, skipping the legs already recorded.

That design only works if re-execution is well behaved, and SPEC §13.7c states the eight properties that say it is:

G1 termination
: propagation halts, whatever the declared depth.

G2 completeness
: a contribution admitted to a dependent survives in every later run until that dependent releases it; there are no orphans.

G3 version stability
: a package released late is released at the version it was originally planned at.

G4 no double ledger delivery
: a contribution does not survive its tag-ledger discharge; this says nothing unconditional about external effects.

G5 no blast-radius widening
: under the retry invariant, later targets are a subset of the first run's.

G6 restricted convergence
: under the retry invariant, repeated successful progress empties the initial plan.

G7 channel discharge
: a prerelease channel is proposed to a package only until it is released on it.

G8 one-way axis independence
: suppressing a version bump does not suppress a channel change; resolved channels may suppress bumps.

1.2 Why formalise them

The specification proves each property briefly, while the algorithm is spread across prose and pseudocode. This report writes the algorithm down once, formally (Sections 2 to 4), and proves the guarantees inside that model (Section 5), along with the determinism and

[1] *Conventional Commits: Monorepo Extension*, version 2.0.0, `specs/ccme-spec/SPEC.md` in the `dispat` repository. Referred to throughout as SPEC, cited by section number.

idempotency obligations of SPEC §17.2 (Section 6), the duality of SPEC §13.7b (Section 7), and the cost bounds and optimisation-equivalence claims of SPEC §§13.11, 20.7 (Section 8).

CCME 2.0.0 states the cross-run retry invariant explicitly and separates the train window from the undelivered window. The proofs below transcribe those conditions. Section 9 preserves the six counterexample classes found in 1.0.0 and records how 2.0.0 resolves each one; none is presented as a current normative defect.

CCME 3.0.0 is the historical first disclosure of a future VCS and rollback design. Released CCME 3.0.1 [2] corrects its algorithm and cost guidance. Dispat 1.8.2 implements neither 3.0 release. The definitions, proofs and E1–E5 experiments concern the implemented CCME 2.0 model; no 3.0 property is used as evidence for them.

1.3 Scope, and what this is not

The subject is the release computation of SPEC §13 and the publish protocol of SPEC §19, which are joint: the guarantees are properties of the two together, and no separation of them is meaningful. Message syntax (SPEC §4–SPEC §8) is taken as given: units arrive already parsed, with their scope-sets resolved and their corrections applied, which is what SPEC §§13.4, 13.4b deliver to SPEC §13.5 onwards. The one exception is Proposition 77, which states the parser's linearity because SPEC §18.3 depends on it.

Four things a complete account would also cover are deliberately outside the model, and no theorem below should be read as saying anything about them: manifest range reconciliation (SPEC §9.4, SPEC §19.5), which happens at publish time and does not feed back into the plan; the diagnostic registry of SPEC §16, of which only the codes naming a suppression are mentioned, as labels; the concurrency and scheduling of the executor, which is an implementation concern that SPEC §17.2 constrains only by forbidding observable non-determinism; and shared-version groups, a feature of the reference implementation (`versioning: fixed`) that SPEC does not define and that changes which packages release together.

The report makes no novelty claim for the underlying patterns: sagas, idempotent consumers and breadth-first search. Its contribution is the formal model, corrected hypotheses, counterexamples and conformance obligations.

1.4 Implementation examples outside the model

Two small implementation examples show how the executor can be composed with other controls; neither extends the theorem scope above. A developer using Windows locally and a Linux CI runner can share one release graph and lifecycle while choosing a shell explicitly. Keep these three files in the repository root:

```
# global.yaml: shared graph, scripts and lifecycle
scripts:
  build: cmake --build build
flow:
  build: [build]
packages:
  app:
    path: app

# linux.yaml: Linux overrides only
$ref: ./global.yaml
shell: ["/bin/sh", "-c"]
```

```
# windows.yaml: Windows overrides only
$ref: ./global.yaml
shell: ["powershell.exe", "-NoProfile", "-NonInteractive",
        "-Command"]
```

The platform files inherit the shared configuration and replace only `shell`. References resolve relative to the file containing them. Overrides replace whole keys, not nested fields: overriding `scripts` would replace that entire map. Individual scripts can instead reference separate files with `$ref`.

From Windows, preview with `dispat -config windows.yaml status`; on the Linux runner, use `dispat -config linux.yaml status`. After its release gates, CI can run `dispat -config linux.yaml` to execute the release. Where a compatible POSIX shell is available, one selector can choose the preview configuration:

```
dispat if OS=Windows_NT \
  --then 'dispat --config windows.yaml status' \
  --else 'dispat --config linux.yaml status'
```

`OS=Windows_NT` tests an environment value; it is not host detection. Here the non-Windows branch assumes a Linux runner. Dispat 1.8.2 executes `if` branches through `/bin/sh -c` without reading configuration; the child invocation then loads its platform shell. Thus the selector needs `/bin/sh`, even though the Windows configuration names PowerShell. A native Windows user can invoke the Windows configuration directly instead. The example assumes CMake is installed and `app/build` is already configured. It does not establish native Windows execution coverage.

A shared `dispat for Debug Release -do "dispat exec build-one"` can likewise pass each build configuration through `DISPAT_ITEM` to a named script. A configuration-free literal `for` loop also requires `/bin/sh`; platform-specific leaf commands must still be valid for the chosen shell. These are composition examples, not additional protocol guarantees.

For an agent-assisted release, complementary gates keep evidence attached to the inputs being published: a lock-free `dispat status -require-release -strict`; an all-package workspace suite with temporary links and cleanup that runs and is verified under `always()`; and package checks after version/lockfile reconciliation plus smoke tests of the exact built artefact before publish. These commands inspect or test inputs; they do not implicitly execute the computed release plan. Every wrapper must preserve exit status, and these checks neither start a release nor change the required authorization policy. HOL Guard PR [10] adds opt-in review when a Dispat release starts, but at its cited open, unmerged revision it is not an implementation or experimental validation of this whole gate stack; that integration remains in progress.

1.5 How to read it

Section 2 is definitional and dense; a reader who wants the results first can read Section 5 directly, following the macro names back to Section 2 as needed. Appendix A is the symbol table. Every definition names the specification section it transcribes, and every theorem names the guarantee it formalises, so the report can be read beside its source. Section 10 maps each result to the conformance vectors of SPEC Appendix B and to the test in the reference implementation that exercises it.

1.6 Licence

SPEC is licensed GPL-3.0-or-later, and this report reproduces its definitions and pseudocode; it is therefore distributed under the same licence. See `paper/LICENSE`.

2 The model

Everything in this report is stated over the objects defined here. Each definition transcribes a numbered section of SPEC and cites it. Where the transcription makes a choice the source leaves implicit, the choice is flagged in a remark rather than buried.

2.1 History

Definition 1 (commit history, SPEC §12.2)**.** The history is a finite directed acyclic graph on a set of commits $\mathcal{K}$, with an edge from a commit to each of its parents. A distinguished commit $h \in \mathcal{K}$ is *HEAD*. For $c \in \mathcal{K}$, $\mathrm{reach}(c) \subseteq \mathcal{K}$ is the set of commits reachable from c by following parent edges, including c itself. Write $c' \preceq c$ for $c' \in \mathrm{reach}(c)$ ("c' is an ancestor-or-self of c").

h is fixed for the whole report. The retry and convergence guarantees of SPEC §13.7c are stated at a fixed h; advancing it may legitimately add, remove, or redirect work. Structural termination and single-state determinism do not need that restriction.

Remark 2. Ancestry, not commit date or log order, is the only ordering used. This is what makes cancellation deterministic under merges and rebases (SPEC §10.4) and what lets the model ignore clocks entirely.

2.2 Workspace

Definition 3 (workspace graph, SPEC §13.1, SPEC §8.4)**.** $\mathcal{P}$ is the finite set of packages present at h. For a fixed set of edge kinds K (the configured `propagation.kinds`), $E \subseteq \mathcal{P} \times \mathcal{P}$ contains (a, b) when a declares a dependency on b of a kind in K. The *dependents* of p are $\mathrm{dep}(p) = \{d : (d, p) \in E\}$. The graph $(\mathcal{P}, E)$ is acyclic.

Acyclicity is an assumption the model may make because the engine enforces it: a cycle over the propagation or publish-ordering kinds is the repository-scoped error `E200` and aborts the run before any plan is computed. SPEC §13.1 justifies rejecting rather than accommodating cycles: no publish order exists for a cycle, so whichever member goes first declares a range on a version that does not yet exist, and if the run then fails that state is permanent.

Propagation travels *against* the dependency edges, from a changed package to the packages that consume it; that is, along $\mathrm{dep}(\cdot)$.

Example 4 (the running workspace, SPEC §22)**.** The running examples use the six-package workspace shared by many conformance vectors:

$$\mathcal{P} = \{\, \texttt{core},\ \texttt{cli},\ \texttt{ui},\ \texttt{api},\ \texttt{docs-site},\ \texttt{@acme/theme} \,\},$$

$$E = \{\, (\texttt{cli}, \texttt{core}),\ (\texttt{ui}, \texttt{core}),\ (\texttt{api}, \texttt{core}),\ (\texttt{docs-site}, \texttt{ui}),\ (\texttt{@acme/theme}, \texttt{ui}) \,\},$$

drawn in Figure 1, with the initial tag state

$$\sigma_0 = \{\ \texttt{core@1.4.2},\ \texttt{cli@2.0.0},\ \texttt{ui@0.9.1},\ \texttt{api@1.2.0},\ \texttt{@acme/theme@1.0.0}\ \}.$$

`docs-site` has never been released, so it has no baseline; it is private and publishes to an internal registry, which by SPEC §13.10a exempts it from nothing.

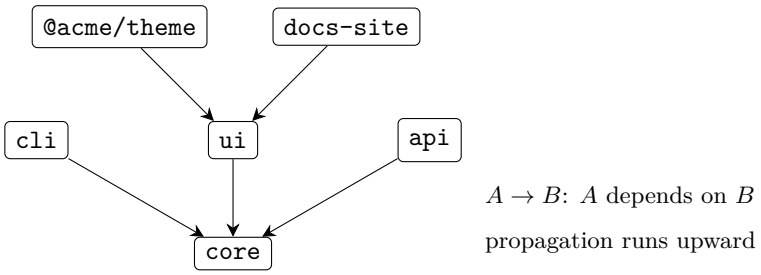


Figure 1: The running workspace of Example 4. Distances from $S = \{\texttt{core}\}$ are $\delta(S, \texttt{cli}) = \delta(S, \texttt{ui}) = \delta(S, \texttt{api}) = 1$ and $\delta(S, \texttt{docs-site}) = \delta(S, \texttt{@acme/theme}) = 2$.

Definition 5 (distance, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $d \in \mathcal{P}$, $\delta(S, d)$ is the least n for which there are $p_0 \in S$ and $p_1, \ldots, p_n = d$ with $p_{i+1} \in \mathrm{dep}(p_i)$ for all i, and ∞ if no such chain exists. Note $\delta(S, d) = 0$ exactly when $d \in S$.

2.3 Bumps and versions

Definition 6 (bumps, SPEC §7.1, SPEC §9.1)**.** $\mathbb{B} = \{\mathsf{none}, \mathsf{patch}, \mathsf{minor}, \mathsf{major}\}$ ordered $\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}$. Being a finite chain, $(\mathbb{B}, \leq)$ is a join-semilattice with least element none and join $\sqcup = \max$.

The whole of SPEC §9.1's "bumps accumulate" is the statement that the combining operator is this join. Two consequences that SPEC states separately are then immediate and are used without further comment: combination is associative, commutative and idempotent, so the order in which contributions are gathered cannot matter; and $x \sqcup y \geq x$, so propagation can only ever raise a package's bump, never lower or replace a direct decision.

Definition 7 (versions and channels, SPEC §11.1–SPEC §11.4)**.** $\mathcal{V}$ is the set of SemVer 2.0.0 versions with SemVer precedence. $\mathcal{C}h = \{\mathsf{stable}\} \cup \mathcal{N}$ for a set $\mathcal{N}$ of channel names. $\mathrm{ch}(v) = \mathsf{stable}$ if v carries no prerelease identifier, and otherwise the first (alphanumeric) prerelease identifier of v. $\mathrm{core}(v)$ is the `major.minor.patch` triple of v and $\mathrm{cnt}(v)$ the numeric counter of a prerelease. $\mathrm{apply}(v, b)$ applies bump b to v, with $\mathrm{apply}(v, \mathsf{none}) = v$.

2.4 State

Everything above is fixed by h and the configuration. Exactly one thing changes from run to run, and isolating it is what makes the guarantees provable as statements about a sequence.

Definition 8 (engine state, SPEC §13.2, SPEC §12.3)**.** A state σ is a finite set of release tags $(p, v, c) \in \mathcal{P} \times \mathcal{V} \times \mathcal{K}$ with $c \preceq h$. States are ordered by inclusion, $\sigma \leq \sigma'$ iff $\sigma \subseteq \sigma'$. From σ:

- $\mathrm{base}_\sigma(p)$ is the highest-precedence version among the tags of p in σ, undefined if there are none;
- $\mathrm{basec}_\sigma(p)$ is the commit of that tag;
- $\mathrm{sbase}_\sigma(p)$ is the highest-precedence non-prerelease version among them, and $\mathrm{scom}_\sigma(p)$ its commit.

Where $\mathrm{base}_\sigma(p)$ is undefined, $\mathrm{ch}(\mathrm{base}_\sigma(p)) = \mathsf{stable}$ by convention (SPEC §11.1): a never-released package is graduated by nothing and is put on a train by any directive naming it.

The baseline functions additionally assume a unique highest-precedence reachable tag for each package and each stable baseline. States with tied precedence and different commits need a

specified tie rule and are outside this proof's domain. Operational lock and temporary refs are not release tags and are not members of σ.

That release-tag states only ever grow is not an assumption about tidy operators; it is normative. SPEC §19.1 forbids moving or deleting a pushed tag and SPEC §12.4 makes tags authoritative over manifests, because a tag is the record that a leg of the transaction completed and an unwritten-back or rewritten record is exactly what makes a saga unrecoverable.

Definition 9 (train and undelivered windows, SPEC §13.3)**.**

$$W_\sigma(p) \;=\; \mathrm{reach}(h) \setminus \mathrm{reach}(\mathrm{scom}_\sigma(p)), \tag{1}$$

and $W_\sigma(p) = \mathrm{reach}(h)$ when p has no stable baseline in σ. The undelivered window is

$$W^\circ_\sigma(p) = W_\sigma(p) \setminus \mathrm{reach}(\mathrm{basec}_\sigma(p)),$$

with an empty subtracted set when p has no baseline tag.

$W_\sigma(p)$ is the train window used to aggregate a prerelease target from its stable base. $W^\circ_\sigma(p)$ is the fresh-admission window for direct work, corrections, channels, and propagation to dependent p. For a stable baseline they coincide; on a train they must not be conflated.

Example 10 (windows)**.** Let the history be $c_0 \to c_1 \to c_2 = h$ with σ_0 of Example 4, where the tags sit at c_0. Then

$$W_{\sigma_0}(\texttt{core}) = \{c_1, c_2\}, \qquad W_{\sigma_0}(\texttt{docs-site}) = \{c_0, c_1, c_2\},$$

the second because a package with no stable baseline has the whole reachable history pending: SPEC §12.5 makes an unreleased package's first release account for everything, which is also why adopting the engine on an existing repository does not need a migration step.

Remark 11 (the window is measured from the last *stable* tag). This is deliberate and it serves SPEC §11.4: a package on a prerelease train must accumulate every commit since its last *stable* release, because the train's target version is recomputed from the stable baseline on every run so that a breaking change arriving mid-train moves the whole train. The consequence, which Section 9 returns to at length, is that a commit already published in a prerelease of p is still a member of $W_\sigma(p)$.

2.5 Units

Definition 12 (units, SPEC §4.4, SPEC §13.4, SPEC §13.4b)**.** $\mathcal{U}$ is the set of units carried by commits in $\bigcup_p W_\sigma(p)$, after parsing (SPEC §20), scope resolution (SPEC §6) and the application of corrections (SPEC §13.4b). For $u \in \mathcal{U}$: $c_u \in \mathcal{K}$ is its commit; $\mathrm{scope}(u) \subseteq \mathcal{P}$ its resolved scope-set; $\mathrm{bump}(u) \in \mathbb{B}$ the bump its type and `!` map to; and its directives are

$$\mathrm{pb}(u) \in \mathbb{B}, \quad \mathrm{pd}(u) \in \mathbb{N} \cup \{\infty\}, \quad \mathrm{ps}(u) \subseteq \mathcal{P}, \quad \mathrm{pc}(u), \quad \mathrm{pcd}(u), \quad \mathrm{pcs}(u) \subseteq \mathcal{P},$$

where $\mathrm{pb}(u)$ is the *effective* propagated bump: the value the unit states, or $\mathrm{bump}(u)$ where it states `inherit`. The two axes default differently and the asymmetry is deliberate (SPEC §8.1). `Propagate` defaults to `patch`, not to `inherit`: a dependent reached by propagation has had a dependency change under it but no change to its own public API, so `patch` is the honest signal, and `feat(core)^` therefore gives its consumers a patch rather than a minor. `Propagate-Channel` defaults to `inherit`, since a channel is not a quantity that can be understated. Both defaults apply only once the corresponding depth is non-zero, and both depths default to 0 (SPEC §8.3, SPEC §8.3a): a unit that says nothing releases its own packages and nothing else. $\mathrm{pc}(u)$ is a channel value: a name, `stable`, `inherit`, `none`, or a transition $\langle f \to t \rangle$.

Taking corrections as already applied is what SPEC §13.4b prescribes: it requires `Edits` and `Deletes` to be resolved into the tuple stream before cancellation runs, and notes that every later phase operates on the corrected stream, which downstream phases cannot distinguish from one that was authored that way.

Remark 13 (the corrected stream depends on σ). It does not follow, and SPEC §13.4b does not claim, that the stream is independent of the state. A correction reaches only *undischarged* work: it applies to a (package, unit) pair only while that package's window still contains the target commit, and once the package has released the target the correction is a no-op for it (`W209`, SPEC §7.4.2). So $\mathcal{U}$ is properly a function $\mathcal{U}_\sigma$, and a record suppressed by a `Deletes` for a package returns to the stream for that package once the package releases. This matters in exactly one place, and it is the same place that a lapsed cancellation matters: proof obligation 80. Everywhere else the two readings coincide, because a released package's own window is empty and its direct contribution is therefore already spent, so the notation $\mathcal{U}$ is kept unindexed below.

Definition 14 (suppression and source packages, SPEC §13.4a)**.** $\mathrm{canc}(c, p)$ holds when c is an ancestor-or-self of some `cancel` commit whose resolved scope contains p (SPEC §10.3); $\mathrm{held}(p)$ holds when p's effective `Release-As` directive is `none` (SPEC §13.6a). Both are computed from the corrected stream, h, configuration, and release-tag state. They are constant across retries only where the retry invariant says those inputs remain fixed. Write

$$\mathrm{disch}_\sigma(p, c) \;\equiv\; c \notin W^\circ_\sigma(p),$$

and then

$$\mathrm{src}_\sigma(u) \;=\; \{\, p \in \mathrm{scope}(u) \;:\; \mathrm{disch}_\sigma(p, c_u) \;\vee\; \neg(\mathrm{canc}(c_u, p) \vee \mathrm{held}(p)) \,\}.$$

The disjunction is the normative content of SPEC §13.4a: suppression applies only to *undischarged* work. Once p has published the version carrying u, the artefact its consumers are owed is public, and neither a later `cancel` (a no-op, `W170`) nor a later hold may retroactively strand those consumers. Proof obligation 80 shows that this clause, and not the traversal, is where **G5** is decided.

2.6 Runs

Definition 15 (plan)**.** Given σ, the release computation of SPEC §13 yields the bumps $\mathrm{dir}_\sigma(p)$ and $\mathrm{prp}_\sigma(p)$ (Definition 29), the channels $\mathrm{chan}_\sigma(p)$ (Definition 25), and $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$. The plan is

$$\begin{aligned}\Pi(\sigma) \;=\; \{\, p \in \mathcal{P} \;:\; & \neg\mathrm{held}(p) \;\wedge\; \big(\mathrm{eff}_\sigma(p) \neq \mathsf{none} \\ & \vee\; \mathrm{chan}_\sigma(p) \neq \mathrm{ch}(\mathrm{base}_\sigma(p)) \;\vee\; p \text{ has an applicable exact } \texttt{Release-As} \\ & \qquad \text{in } W^\circ_\sigma(p)) \,\},\end{aligned}$$

each $p \in \Pi(\sigma)$ carrying the version $\nu_\sigma(p)$ of SPEC §13.9. The fresh bumps above decide whether work is newly admissible; they are not by themselves the prerelease target. On a train, the intended calculation aggregates the surviving corrected units in $W_\sigma(p)$ from the stable baseline, including work already delivered to an earlier prerelease. For graduation, the model applies the specific rule of SPEC §11.5: publish that same train target without a prerelease suffix. The generic summary in SPEC §13.9 is inconsistent with this rule; proof obligation 78 records the textual obligation.

Remark 16 ($\Pi(\sigma)$ is the released set, not the printed one). SPEC §13.9 computes a would-be version for a held package too, and SPEC §13.6a requires the engine to report it (`W154`) so that the number needed to lift the hold is available without hand computation. That reported entry is part of what an operator calls "the plan", but it is not a release: SPEC §13.10 excludes it. $\Pi(\sigma)$ above is the released set, which is why Theorem 55 needs no exception for holds and why the convergence check of SPEC §19.6 states one: the two are reading the same word differently, and only SPEC §19.6's reading has to be told about held packages.

Definition 17 (run, SPEC §19.1–SPEC §19.3)**.** A run from σ is parameterised by a set $F \subseteq \Pi(\sigma)$ of publishes that fail. It induces the blocked set

$$B \;=\; \{\, p \in \Pi(\sigma) \setminus F \;:\; p \text{ transitively depends on some } q \in F \text{ over } \texttt{publish.blockingKinds} \,\},$$

publishes $\Pi(\sigma) \setminus (F \cup B)$ in the order of SPEC §19.2, and yields

$$\mathrm{run}(\sigma, F) \;=\; \sigma \;\cup\; \{\, (p, \nu_\sigma(p), h) \;:\; p \in \Pi(\sigma) \setminus (F \cup B) \,\}.$$

Three features of this definition carry weight later.

F is chosen adversarially and independently on each run. The guarantees are claims about *every* failure schedule, not about a well-behaved one; a proof that appeals to what usually fails proves nothing about a system whose purpose is to survive what does not usually happen.

Every tag is written at h, never at the commit the work arrived in (SPEC §19.1). This is what makes Lemma 40 a two-case statement rather than an argument about which commits a tag covers.

A package is tagged if it is released, whatever its publish target, including targets of `none` that upload nothing (SPEC §13.10a). Proposition 56 shows this is forced: the alternative is a permanent hole in **G6**.

Definition 18 (run sequence)**.** A *run sequence* at h is $\sigma_1 \le \sigma_2 \le \cdots$ with $\sigma_{k+1} = \mathrm{run}(\sigma_k, F_k)$ for some $F_k \subseteq \Pi(\sigma_k)$. By Definition 17 it is monotone: $\sigma_k \subseteq \sigma_{k+1}$.

3 Propagation

This section transcribes SPEC §9.2, the three-phase procedure that assigns each package a propagated channel, resolves every package's channel, and then assigns propagated bumps. The phase order is forced: phase 3 reads what phase 2 produces, and SPEC §9.2 forbids fusing them.

3.1 The traversal

Definition 19 (reach, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $\ell \in \mathbb{N} \cup \{\infty\}$,

$$R(S, \ell) \;=\; \{\, d \in \mathcal{P} \setminus S \;:\; \delta(S, d) \le \ell \,\},$$

and each $d \in R(S, \ell)$ carries the level $\delta(S, d)$.

```
reach(edges, sources, depth):
    frontier = sources
    seen     = set(sources)      # a unit never reaches its own packages
    out      = {}                # package -> level
```

```
    for level in 1 .. depth:     # 'all' => until frontier is empty
        next = {}
        for p in frontier:
            for d in edges.dependentsOf(p):
                if d in seen: continue
                seen.add(d); next.add(d)
        if next is empty: break
        for d in next: out[d] = level
        frontier = next
    return out
```

Lemma 20 (the traversal computes Definition 19)**.** *On termination,* `out` *has domain* $R(S, \ell)$ *and* `out`$[d] = \delta(S, d)$.

Proof. Standard breadth-first search from the set S, seeded at level 0. By induction on `level`: `frontier` at the start of iteration n is exactly $\{d : \delta(S, d) = n - 1\} \setminus S$ for $n > 1$ and S for $n = 1$, because `seen` contains precisely the packages at distance $\leq n - 1$ (together with S at level zero), and a package first enqueued at iteration n has a chain of length n from S and none shorter. Membership of `seen` is tested before insertion and never cleared, so each package is assigned at most one level, its least. The loop runs for ℓ iterations or until the frontier is empty. Seeding `seen` $= S$ removes S from the output, which is Definition 19's $\mathcal{P} \setminus S$. □

Three properties recorded by SPEC §9.2 are visible in this statement and are used repeatedly below. Depth is shortest-path, so a package reachable at both depth 1 and depth 3 is included by `+1`. Depth is measured from the originating source set as a whole and never recomputed from an intermediate package, so a unit written `+1` reaches exactly the direct consumers of its own packages in every run, whatever released in between. And a package reached by several paths is visited once, so a diamond does not double-count.

Example 21 (depth, on Figure 1)**.** Take $S = \{$`core`$\}$, the source set of a unit scoped `(core)`:

Written	$R(S, \ell)$	Note
`feat(core)`	$\emptyset$	$\ell = 0$, the default
`feat(core)^`	{`cli`, `ui`, `api`}	direct consumers
`feat(core)+2`	$R(S, 1) \cup$ {`docs-site`, `@acme/theme`}	two edges out
`feat(core)^^`	the same set	$\ell = \infty$; the graph is only two deep

Now take the two-package source set $S' = \{$`core`, `ui`$\}$, as a unit scoped `(core,ui)` would have. Then $R(S', 1) = \{$`cli`, `api`, `docs-site`, `@acme/theme`$\}$: `ui` is *absent* although it is a dependent of `core` at distance 1, because Definition 19 removes the whole of S'. The unit bumps `ui` directly, and a package is never both a direct and a propagated target of the same unit. Proposition 72 is about exactly this clause.

Remark 22 (self-exclusion is per unit)*.* $R(S, \ell)$ excludes all of S, not merely the source a given path started from. This is what stops a unit from propagating to a package it already bumps directly. It is also the clause that makes the bucketed traversal of SPEC §13.11 unsound if transcribed naively; see Proposition 72.

3.2 Phase 1: the channel axis

Definition 23 (proposed channel, SPEC §9.3)**.** For a unit u and package d, let $cur = \mathrm{ch}(\mathrm{base}_\sigma(d))$ (stable if d has no baseline). Then $\mathrm{pcf}_\sigma(u,d) \in \mathcal{C}h \cup \{\bot\}$ is:

- if $\mathrm{pc}(u)$ is a transition $\langle f \to t\rangle$: $\bot$ if $f = t$ (`W207`); $\bot$ if cur does not match f, where $f = *$ matches any $cur \neq$ stable and otherwise f matches only $cur = f$; else $target = t$;
- otherwise $target$ is $\mathrm{pc}(u)$, or the origin channel of u where that is `inherit`; and if $target =$ stable while $cur \neq$ stable, the result is $\bot$ (`W200`: a propagated channel never graduates implicitly);
- in either case, if $target = cur$ the result is $\bot$ (`W199`); otherwise it is $target$.

The origin channel of u is the channel u's own `Channel` directive assigns to its sources if it carries one, and otherwise $\mathrm{ch}(\mathrm{base}_\sigma(p))$ for its sources p, taking the value of the byte-wise least name if they disagree (`W160`).

The final clause complements the target $W^\circ(a)$dmission gate: it compares $target$ against the package's *baseline* channel rather than against anything computed in the current run. This no-op check is the other premise of **G7**, and Theorem 57 is its consequence.

Definition 24 (phase 1 output, SPEC §9.2)**.** $\mathrm{prop}\chi_\sigma(d)$ is the value $\mathrm{pcf}_\sigma(u,d)$ of the $\prec$-greatest unit u (newest commit first, then last unit within a commit, SPEC §11.6) satisfying: $\mathrm{pcd}(u) \neq 0$; $\mathrm{pc}(u) \neq$ `none`; $\mathrm{src}_\sigma(u) \neq \emptyset$; $d \in R(\mathrm{src}_\sigma(u), \mathrm{pcd}(u))$; $d \in \mathrm{pcs}(u)$; $c_u \in W^\circ_\sigma(d)$; $\neg\mathrm{canc}(c_u, d)$; and $\mathrm{pcf}_\sigma(u,d) \neq \bot$. It is undefined if no unit qualifies.

3.3 Phase 2: channel resolution

Definition 25 (channel resolution, SPEC §13.8)**.** For every $p \in \mathcal{P}$, whether or not it will be released, let $base = \mathrm{ch}(\mathrm{base}_\sigma(p))$ and let $D_\sigma(p)$ be the value proposed by the $\prec$-greatest unit u with $p \in \mathrm{scope}(u)$, $c_u \in W^\circ_\sigma(p)$, $\neg\mathrm{canc}(c_u, p)$, that carries a `Channel` directive proposing something for p (a transition matching $base$, or a value differing from $base$; a directive proposing nothing is not a competitor and does not count towards `W186`). Then

$$\mathrm{chan}_\sigma(p) \;=\; \begin{cases} D_\sigma(p) & \text{if defined,} \\ \mathrm{prop}\chi_\sigma(p) & \text{else if defined,} \\ base & \text{otherwise.} \end{cases}$$

A direct directive beats every propagated one regardless of age; among direct directives, and separately among propagated ones, the newest commit wins and then the last unit within it. A package named by nothing keeps its baseline channel, which is why an established train needs no directives to stay together.

3.4 Phase 3: the bump axis

Definition 26 (resolvability, SPEC §9.3a)**.** $\mathrm{sch}_\sigma(u) = \{\mathrm{chan}_\sigma(p) : p \in \mathrm{src}_\sigma(u)\}$ and

$$\mathrm{rsv}_\sigma(u,d) \;\equiv\; \text{stable} \in \mathrm{sch}_\sigma(u) \;\vee\; \mathrm{chan}_\sigma(d) \in \mathrm{sch}_\sigma(u).$$

A bump is a claim that the dependent has something new to pick up, and across a channel boundary the claim is false: a stable consumer resolves a beta-releasing dependency by its stable range exactly as before, so a republished consumer would contain byte-for-byte what its predecessor contained. Suppression is reported per suppressed pair as `W208`.

Example 27 (a caret that reaches and does not bump: SPEC vector 95)**.** The unit `feat(core)^%beta: x` on Figure 1. The `%` puts `core` on `beta`; the `^` reaches $\{\texttt{cli}, \texttt{ui}, \texttt{api}\}$. Phase 2 leaves the three consumers where their baselines put them, on stable, so

$$\mathrm{sch}_\sigma(u) = \{\texttt{beta}\}, \qquad \mathsf{stable} \notin \mathrm{sch}_\sigma(u), \qquad \mathrm{chan}_\sigma(\texttt{cli}) = \mathsf{stable} \notin \mathrm{sch}_\sigma(u),$$

and Item (A6) fails for every one of them: three `W208` diagnostics, and $\Pi(\sigma) = \{\texttt{core}\}$ alone. The caret was honoured and reached its targets; what it would have delivered is not resolvable by them. Writing `feat(core)^%beta++1` instead moves the same three onto `beta` in phase 1, so $\mathrm{chan}_\sigma(\texttt{cli}) = \texttt{beta} \in \mathrm{sch}_\sigma(u)$ and all four packages release, which is the form that keeps a train installable as a set.

Definition 28 (admission, SPEC §9.2, SPEC §13.4a)**.** For $u \in \mathcal{U}$ and $d \in \mathcal{P}$, $\mathrm{adm}_\sigma(u, d)$ holds iff all of:

(A1) $\mathrm{bump}(u) \neq \mathsf{none}$, $\mathrm{pb}(u) \neq \mathsf{none}$, $\mathrm{pd}(u) \neq 0$;
(A2) $\mathrm{src}_\sigma(u) \neq \emptyset$ and $d \in R(\mathrm{src}_\sigma(u), \mathrm{pd}(u))$;
(A3) $d \in \mathrm{ps}(u)$;
(A4) $c_u \in W^\circ_\sigma(d)$;
(A5) $\neg\mathrm{canc}(c_u, d)$;
(A6) $\mathrm{rsv}_\sigma(u, d)$.

Definition 29 (propagated and direct bumps, SPEC §13.6, SPEC §13.7)**.**

$$\mathrm{prp}_\sigma(d) = \bigsqcup\{\, \mathrm{pb}(u) \;:\; u \in \mathcal{U},\ \mathrm{adm}_\sigma(u, d) \,\},$$
$$\mathrm{dir}_\sigma(p) = \bigsqcup\{\, \mathrm{bump}(u) \;:\; p \in \mathrm{src}_\sigma(u) \cap \mathrm{scope}(u),\ c_u \in W^\circ_\sigma(p) \,\},$$

both empty joins being none. Then $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$.

Item (A4) is the load-bearing clause of the entire design, and it is worth being explicit about what it is not. It tests the *target's* window, not the source's. SPEC §13.7a shows what the other reading costs: with retention governed by the source's window, a dependent whose publish failed while its provider's succeeded is silently orphaned, permanently one patch behind a dependency it declares, with no diagnostic, because from the engine's point of view nothing is pending. Every guarantee in Section 5 rests on Item (A4) being the target's window; Theorem 47 is where that is used.

Example 30 (the orphan, in sets: SPEC vector 66)**.** One commit $C = h$ on the workspace of Example 4 carrying the unit $u =$ `feat(core)^: streaming reader`. So $\mathrm{scope}(u) = \{\texttt{core}\}$, $\mathrm{bump}(u) = \mathsf{minor}$, $\mathrm{pb}(u) = \mathsf{patch}$ (the default of Definition 12) and $\mathrm{pd}(u) = 1$.

Run 1 evaluates Definition 28 with σ_0. All of `cli`, `ui`, `api` lie in $R(\{\texttt{core}\}, 1)$, none is cancelled, all are on stable as is `core`, and $C \in W_{\sigma_0}(d)$ for each, so all three are admitted and

$$\Pi(\sigma_0) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}, \qquad \nu_{\sigma_0}(\texttt{core}) = 1.5.0, \quad \nu_{\sigma_0}(\texttt{cli}) = 2.0.1.$$

Suppose `core`, `ui` and `api` publish and `cli` fails, so

$$\sigma_1 = \sigma_0 \cup \{\texttt{core@1.5.0},\ \texttt{ui@0.9.2},\ \texttt{api@1.2.1}\}.$$

Run 2 is where the two readings of Item (A4) separate:

target's window (normative): $C \in W_{\sigma_1}(\texttt{cli}) = \{C\} \;\Rightarrow$ `cli` admitted,
source's window (the trap): $C \notin W_{\sigma_1}(\texttt{core}) = \emptyset \;\Rightarrow\; u$ reaches nobody.

Under the normative rule $\Pi(\sigma_1) = \{\texttt{cli}\}$ at the version run 1 planned, and $\Pi(\sigma_2) = \emptyset$. Under the trap $\Pi(\sigma_1) = \emptyset$ and `cli` is never released, on this run or any later one, with no diagnostic anywhere. One membership test, evaluated against a different package, is the whole difference.

Remark 31 (catch-up is not a mode). There is no separate catch-up pass in this model, and none in the specification. A "catch-up release" is a label for a release whose entire cause is a propagation from a package not itself in the run's plan (`W193`); the engine does nothing different to produce it. It does not widen depth, resurrect a cancelled contribution, release a held package, change a planned version, or propagate onward from the catching-up package; each of which is a property of Definition 28 rather than a rule about catch-up.

4 The publish protocol

The guarantees are joint properties of the computation and the protocol that writes the state the next computation reads. SPEC §19 says so explicitly: they do not survive an implementation that gets this section wrong. Three obligations are used in the proofs.

4.1 Tag after, immediately, and always

Definition 32 (tagging, SPEC §19.1)**.** For each released package: the tag is created at h; *after* that package's publish succeeds, never before; *immediately* after it, not batched to the end of the run; and for every released package whatever its publish target, including targets that upload no artefact at all.

Each clause excludes a specific failure. Tagging before publishing makes the version unrecoverable, because Definition 9 will treat it as released. Batching tags to the end of a run makes a mid-run failure republish everything already published, which most registries reject outright. Tagging only what uploads an artefact is the subject of Proposition 56.

4.2 Order

Definition 33 (publish order, SPEC §19.2)**.** Let G be the workspace graph restricted to `publish.orderKinds` over *all* packages. Publication proceeds in a topological order of G, ties broken by byte-wise least name, filtered to the plan at the end.

Proposition 34 (the order exists, is total, and is deterministic)**.** *Definition 33 is well defined and yields the same sequence on every machine and in every conforming implementation.*

Proof. G is acyclic by Definition 3, so a topological order exists. Ties are broken by a total order on names (byte-wise comparison of distinct names), so the order is total and independent of any container's iteration order. The filter is applied to a sequence, preserving it. □

Remark 35 (order over the whole graph, then filter). Inducing the graph on the planned packages *first* is a defect, not an optimisation. With $A \to B \to C$ where B has no bump this run (the common case in any incremental release), the induced subgraph on $\{A, C\}$ has no edge at all, so A may publish against a C that has not published yet. The constraint is real even though B is not released, because A still resolves C through B at install time. This is why the model orders over $\mathcal{P}$ and filters last, and why an implementation detecting a violation must raise `E197`.

Order matters for a reason no later run can repair. A dependent published before its dependency carries a range reconciled against the dependency's old version, and both land at the same commit, so no ancestry test can tell afterwards that anything is wrong (SPEC §13.7b). The inconsistency is undetectable after the fact, which is why it is prevented rather than diagnosed.

4.3 Partial failure

Definition 36 (blocking, SPEC §19.3)**.** On a failure, every package in the plan that transitively depends on the failed one over `publish.blockingKinds` is *blocked* (`W194`): planned, but not attempted in this run. The closure is computed over the full workspace graph, so it traverses packages that are not in the plan. Packages that are not blocked continue to publish.

Ordering and blocking use different edge-kind sets, and the asymmetry is deliberate: `orderKinds` includes optional dependencies, because an optional dependency should still be available before its dependent, while `blockingKinds` excludes them, because a missing optional dependency does not break an install and a missing required one does.

Example 37 (an infrastructure leg)**.** Nothing in Definitions 17 and 32 requires a publish leg to be a registry upload. A `terraform apply` may be such a leg: its build stage imports the explicitly configured live resources and writes a plan, and its publish stage applies that plan before the release tag is written. The tag records that this version of the configuration completed successfully. It is not Terraform resource state, does not prove that every live resource was imported, and does not make applying an arbitrary configuration externally idempotent. Application builds may overlap the apply; only their publication waits for the infrastructure publish edge.

Remark 38 (recorded-success abstraction). Definition 17 models confirmed successful legs by their tags. It proves properties of that release ledger, not convergence of an external registry or infrastructure state. A publish may take effect before its tag is recorded; then the repository alone is insufficient and Definition 39 or a publish-target-specific idempotence contract is required. Terraform likewise needs its own state or an explicit, complete import boundary before a plan can safely describe existing resources.

Definition 39 (reconciliation, SPEC §19.4)**.** When a publish fails because the version already exists, the implementation verifies by registry digest or provenance attestation that the published artefact is the one this run would have published; if it matches, the publish counts as successful and the tag is written (`W196`); if it does not match, or identity cannot be established, the run stops with `E198`.

This is the one place where the model's "a publish either succeeds or fails" abstraction meets a third outcome, that it already happened, and Definition 39 maps it back onto success. Verification is what makes that safe: without it, adopting a version published by something other than this run would enrol a foreign artefact into the release lineage, which is a supply-chain vulnerability rather than a convenience.

5 The eight guarantees

Throughout this section, h and the configuration are fixed and $\sigma_1 \le \sigma_2 \le \cdots$ is a run sequence (Definition 18). Two hypotheses are stated once and used by name.

(H1) stable inherited-channel state
every baseline channel read by `Propagate-Channel: inherit` remains unchanged through the sequence.

(H2) no suppressed source is released
for every unit u and every $p \in \mathrm{scope}(u)$ whose contribution to u is suppressed at σ_1, whether by cancellation, by a hold (Definition 14), or by a correction (Remark 13), p is not released at h in the sequence.

The *retry invariant* of CCME 2.0.0 additionally fixes, for every outstanding unit, the corrected tuple stream, source sets, graph, scope and depth predicates, target baselines, and resolved source/target channel admission. The only permitted state changes are baseline tags written for packages that successfully release. Thus it entails the constancies used below; (H1) and (H2) name two historically problematic components, not a weaker substitute for the full invariant.

Both clauses are explicit parts of CCME 2.0.0's retry invariant. They are named because the 1.0.0 proof audit isolated them in counterexamples (proof obligations 80 and 83); they are not sufficient on their own. The 2.0.0 fresh window resolves the prerelease-discharge defect of proof obligation 79, while retries still require every other input listed above to remain fixed. Every theorem below states when it needs that full invariant.

5.1 Three lemmas

Lemma 40 (window antitonicity)**.** *Let $\sigma' = \mathrm{run}(\sigma, F)$. For every $p \in \mathcal{P}$:*

$$W_{\sigma'}(p) = \begin{cases} \emptyset & \text{if } p \text{ published a non-prerelease version in the run,} \\ W_\sigma(p) & \text{otherwise.} \end{cases}$$

In particular $W_{\sigma'}(p) \subseteq W_\sigma(p)$, and along a run sequence $W_{\sigma_j}(p) \subseteq W_{\sigma_k}(p)$ for $j \geq k$. For the undelivered window, every successful release of p, stable or prerelease, gives $W^\circ_{\sigma'}(p) = \emptyset$; otherwise it is unchanged.

Proof. By Definition 17 the run adds only tags of the form $(p, \nu_\sigma(p), h)$. If p is not among them, its tag set is unchanged, hence so are $\mathrm{scom}_\sigma(p)$ and its window. Suppose p is among them and $\nu_\sigma(p)$ is not a prerelease. Then $\nu_\sigma(p)$ exceeds $\mathrm{base}_\sigma(p)$ in SemVer precedence, and therefore exceeds $\mathrm{sbase}_\sigma(p)$: SPEC §13.9 computes it as $\mathrm{apply}(\mathrm{sbase}_\sigma(p), b)$ for the graduation and stable cases, and the guards `E185`, `E195`, `E153` and `E156` reject any computed or pinned version that fails to exceed the baseline. So the new tag is the highest-precedence non-prerelease tag of p, giving $\mathrm{scom}_{\sigma'}(p) = h$ and $W_{\sigma'}(p) = \mathrm{reach}(h) \setminus \mathrm{reach}(h) = \emptyset$. If $\nu_\sigma(p)$ is a prerelease, it is not a candidate for sbase() at all, so scom() and the train window are unchanged. Its tag is nevertheless the new baseline at h, so $W^\circ()$ is empty. The final claim follows by induction along the sequence. □

Remark 41. The third case of the proof, a prerelease release leaving the window untouched, is not an oversight in Definition 9 but its stated purpose (Remark 11). Admission nevertheless uses the empty undelivered window. Proof obligation 79 records the 1.0.0 defect that motivated this separation.

Lemma 42 (outstanding-input stability)**.** *Assume the retry invariant. For every outstanding unit u and unreleased target d, its corrected tuple, source set, graph and scope/depth predicates, target baseline, resolved source and target channels, and channel-eligibility decision are the same in every retry.*

Proof. This is the retry invariant specialised to one unit and target. H1 prevents an inherited baseline channel from drifting; H2 prevents a successful source release from changing discharge and hence the source set. The remaining constancies are separate clauses of the invariant and cannot be derived from H1 and H2 alone. □

Lemma 43 (window restriction)**.** *Under the retry invariant, admission of an outstanding unit at an unreleased target can change only by restriction of the target's undelivered window. A successful release of the target empties that window.*

Proof. All other admission inputs are fixed by the retry invariant. The first assertion then follows from Definition 28; the second is Lemma 40. □

Remark 44 (why H1 is part of the invariant). Without H1, the origin channel of an `inherit` unit can change when a source baseline moves from a train to stable. That can change an unreleased dependent's resolved channel and planned version between retries. Proof obligation 83 preserves the 1.0.0 counterexample; CCME 2.0.0 prevents it by fixing inherited baseline-channel state in the retry invariant.

5.2 G1: termination

Theorem 45 (**G1**, termination)**.** *Every traversal of Definition 19 halts after at most* $|\mathcal{P}|$ *insertions into* `seen` *and* $O(|\mathcal{P}| + |E|)$ *work, regardless of the declared depth, including* ∞ *(`+*`), and regardless of whether the graph is acyclic. Propagation performs at most* $2|\mathcal{U}|$ *such traversals, so it halts, in* $O(|\mathcal{U}| \cdot (|\mathcal{P}| + |E|))$.

Proof. A package is inserted into `seen` only after a test that it is absent, and `seen` is never cleared, so there are at most $|\mathcal{P}|$ insertions. Only inserted packages (and the initial sources, inserted at seeding) ever enter `next` and hence `frontier`, so each package is expanded at most once, and the edges scanned across the whole traversal are therefore at most $\sum_p |\mathrm{dep}(p)| = |E|$. The loop terminates either by exhausting ℓ iterations or, when $\ell = \infty$, by the empty-frontier break, which must occur within $|\mathcal{P}|$ iterations since each non-empty frontier consumes at least one distinct unvisited package. No step of this argument uses acyclicity: a cycle merely means some edge leads to a package already in `seen`, and that edge is skipped. The outer loops of SPEC §9.2 run over the finite set $\mathcal{U}$ once per axis. □

Remark 46. Acyclicity (Definition 3) is therefore not what makes propagation terminate; `seen` is. Acyclicity is needed for the publish order (Proposition 34), not here. This matters because it means an implementation that loosens the `E200` check does not silently gain a non-terminating traversal; it gains an unorderable publish phase, which is a different and louder failure.

5.3 G2: completeness

Theorem 47 (**G2**, completeness: no orphans)**.** *Assume the retry invariant. If* $\mathrm{adm}_{\sigma_1}(u, d)$*, then for every* j*, either* d *was released in some run* $i < j$*, or* $\mathrm{adm}_{\sigma_j}(u, d)$*. Consequently* $\mathrm{prp}_{\sigma_j}(d) \geq \mathrm{pb}(u)$ *in every run until* d *releases at a commit whose ancestry contains* c_u*.*

Proof. Suppose d has not been released in runs $1..j-1$. Take the conjuncts of Definition 28 in turn. Item (A1) depends only on the unit. Item (A3) and Item (A5) depend only on h and the configuration. Item (A2) is $d \in R(\mathrm{src}_{\sigma_j}(u), \mathrm{pd}(u))$, and by the retry invariant the source set is unchanged, as are the graph and the depth, so the traversal is literally the same computation. Item (A4) is $c_u \in W^{\circ}_{\sigma_j}(d)$; d gained no tag, so by Lemma 40 its window is unchanged. Item (A6) is constant by Lemma 43. So every conjunct still holds.

For the final claim: $\mathrm{prp}_{\sigma_j}(d)$ is a join over the admitted units (Definition 29) and u is among them, so the join is at least $\mathrm{pb}(u)$. The only way Item (A4) can fail is for c_u to leave $W^{\circ}_{\sigma_j}(d)$, which by Lemma 40 requires d to be released at h; and $c_u \preceq h$, so that release carries c_u in its ancestry. □

The theorem is the formal content of "there are no orphans". What it excludes is the failure SPEC §13.7a describes: a provider publishes, a consumer's publish fails, and the consumer is

thereafter permanently one patch behind a dependency it declares, with its manifest range never reconciled and no diagnostic fired, because nothing appears to be pending. Note where the proof would break under the other reading of Item (A4). If admission tested $c_u \in W_{\sigma_j}(p)$ for the unit's own package p, then p's successful release would empty *that* window, the unit would cease to be a propagation source, and d would never be released, in this run or any later one. Nothing else in the model changes; that one substitution is the whole difference between a resumable release and a silently lossy one.

5.4 G3: version stability

Theorem 48 (**G3**, version stability)**.** *Assume the retry invariant. If $d \in \Pi(\sigma_1)$ and d is not released in runs $1..k-1$, then $d \in \Pi(\sigma_k)$ and $\nu_{\sigma_k}(d) = \nu_{\sigma_1}(d)$.*

Proof. d gained no tag, so $\mathrm{base}_{\sigma_k}(d) = \mathrm{base}_{\sigma_1}(d)$ and $\mathrm{sbase}_{\sigma_k}(d) = \mathrm{sbase}_{\sigma_1}(d)$, and by Lemma 40 its window is unchanged. Then $\mathrm{dir}_{\sigma_k}(d) = \mathrm{dir}_{\sigma_1}(d)$, because Definition 29 reads the window, $\mathrm{scope}(u)$, and $\mathrm{src}_\sigma(u)$, the last constant by the retry invariant. And $\mathrm{prp}_{\sigma_k}(d) = \mathrm{prp}_{\sigma_1}(d)$: the $\geq$ direction is Theorem 47 applied to each unit admitted in run 1, and the $\leq$ direction is Theorem 53. Hence $\mathrm{eff}_{\sigma_k}(d) = \mathrm{eff}_{\sigma_1}(d)$. Its resolved and baseline channels, changed-provider status, and applicable exact `Release-As` cause are also fixed by the retry invariant. Thus whichever disjunct of Definition 15 put d in the initial plan still holds. For a fixed version function, SPEC §13.9 reads unchanged stable and current baselines, resolved channel, applicable exact version, and corrected train records. Those inputs are fixed for an unreleased target by the retry invariant, so its result is unchanged. This proves replay stability. The graduation core itself follows the specific train-target rule of SPEC §11.5; proof obligation 78 records the inconsistent SPEC §13.9 summary rather than treating that intent as absent. □

Remark 49 (the prerelease case survives, for a different reason)*.* For a target d planned on a prerelease channel, the version is core$(\cdot)$-plus-counter per SPEC §11.4, and the counter is read from $\mathrm{base}_\sigma(d)$, which is unchanged precisely because d was *not* released. This preserves the counter, but extending Theorem 48 to prerelease targets still requires invariant resolved channels and admitted contributions, including H1's channel-stability component. The prerelease problem of proof obligation 79 is about packages that *were* released, and is a convergence failure, not a stability failure.

Theorem 48 is as much an operational property as a formal one: after a partial failure, the re-run publishes the same numbers the operator already reviewed. It is what makes a resumed release a continuation rather than a fresh decision.

Example 50 (four theorems in one table)**.** Table 1 continues Example 30 through an adversarial schedule: run 1 loses `cli`, run 2 loses it again, run 3 succeeds. Each column is a theorem. `docs-site` and `@acme/theme` never appear, at any distance 2, in any run: the unit said `^`, and Theorem 53 forbids a later run from reconsidering that.

5.5 G4: no double release

Theorem 51 (**G4**, no double ledger delivery)**.** *If d is released in run k at a commit whose ancestry contains c_u, then $\mathrm{adm}_{\sigma_j}(u, d)$ is false for every $j > k$.*

Proof. The successful tag is at h and contains c_u in its ancestry, so $c_u \notin W^\circ_{\sigma_j}(d)$. Clause (A4) of admission therefore fails. The full train window may retain c_u for aggregate prerelease version arithmetic; it cannot re-admit the contribution. □

Run	State σ	Plan $\Pi(\sigma)$	Published	Empty?
1	σ_0 (Example 4)	`core` 1.5.0, `cli` 2.0.1, `ui` 0.9.2, `api` 1.2.1	`core`, `ui`, `api`; `cli` fails	no
2	$\sigma_1 = \sigma_0 \cup 3$ tags	`cli` 2.0.1	nothing; `cli` fails again	no
3	$\sigma_2 = \sigma_1$	`cli` 2.0.1	`cli`	no
4	$\sigma_3 = \sigma_2 \cup \{$`cli@2.0.1`$\}$	$\emptyset$	—	yes

Table 1: The run sequence of Example 50. Theorem 47: `cli` is in every plan until it publishes. Theorem 48: it is at `2.0.1` in all of them. Theorem 51: `core`, `ui` and `api` leave after run 1. Theorem 53: nothing new ever enters. Theorem 55: the plan shrinks to empty. Run 2 publishes nothing, which Theorem 55 treats as the hard-error case rather than the partial-failure one.

Corollary 52 (exactly-once ledger discharge)**.** *Under the retry invariant, a contribution admitted to d remains outstanding until d releases (Theorem 47) and cannot be delivered in the tag ledger again after that release (Theorem 51). If retries eventually publish d, it is therefore discharged exactly once in that ledger.*

This is the idempotent-consumer property of the saga, obtained without a deduplication table: the git tag is the record, and the window is the query against it. This is not an exactly-once guarantee for external side effects. A publish can succeed before its tag is recorded; recovery across that gap requires the verified adoption of SPEC §19.4 or an idempotent publisher. The ledger proof alone cannot prevent a repeated external operation.

5.6 G5: no blast-radius widening

Theorem 53 (**G5**, no blast-radius widening)**.** *Assume the retry invariant. For every unit u and every $j \geq 1$,*

$$\{\, d : \mathrm{adm}_{\sigma_j}(u, d) \,\} \;\subseteq\; \{\, d : \mathrm{adm}_{\sigma_1}(u, d) \,\}.$$

Proof. Compare the conjuncts of Definition 28, as in Theorem 47, but now in the direction of restriction. Items (A1), (A3) and (A5) are constant. Item (A2) is constant by the retry invariant: the traversal has the same source set, the same depth and the same graph, all fixed at h. Item (A6) is constant under the retry invariant; for a target that has been released Item (A4) fails by Lemma 40. Item (A4) is antitone by Lemma 40. A conjunction of constant and antitone predicates is antitone. □

Remark 54 (why this is a security property). SPEC §18.1 depends on Theorem 53. If the admitted set could grow between runs, an attacker able to induce a publish failure (by exhausting a registry quota, say) could enlarge the set of packages a commit releases *after* that commit had been reviewed and merged. The theorem says the achievable effect of inducing a failure is confined to delaying a subset of what was already going to be released.

5.7 G6: convergence

Theorem 55 (**G6**, convergence)**.** *Assume the retry invariant, and suppose every run whose plan is nonempty publishes at least one package. Let $n = |\Pi(\sigma_1)|$. Then $\Pi(\sigma_{n+1}) = \emptyset$.*

Proof. First, $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$. Take $p \in \Pi(\sigma_{j+1})$. If p was released in run j, its new baseline at h empties $W^\circ_{\sigma_{j+1}}(p)$, so no old direct, propagated, channel, or exact-version cause can re-admit it under the invariant. Thus it was not released, and by Lemma 40 its window is unchanged and its tags are unchanged. Then $\mathrm{dir}_\sigma(p)$ is unchanged, $\mathrm{prp}_\sigma(p)$ can only have shrunk (Theorem 53 applied to each unit), $\mathrm{chan}_\sigma(p)$ and $\mathrm{ch}(\mathrm{base}_\sigma(p))$ are unchanged under the retry invariant, and the `Release-As` disjunct is unchanged; so every disjunct of Definition 15 that holds at σ_{j+1} held at σ_j, and $\mathrm{held}(p)$ is constant.

Now let $\mu_j = |\Pi(\sigma_j)|$. Run j publishes some $p \in \Pi(\sigma_j)$, whose undelivered work is absent from $\Pi(\sigma_{j+1})$, while by the paragraph above nothing is added. So $\mu_{j+1} < \mu_j$. A strictly decreasing sequence in $\mathbb{N}$ bounded below by 0 reaches 0 within $\mu_1 = n$ steps. □

The bound is restricted to the invariant domain. Publishing a suppressed source or changing channel admission may expose a finite follow-up obligation; SPEC §19.6 requires re-planning and renewed review, not an unconditional global-convergence claim. Held packages remain excluded.

Proposition 56 (universal tagging is necessary, SPEC §13.10a)**.** *Assume the retry invariant. Suppose the protocol were weakened so that a released package p whose publish target uploads no artefact is versioned and has its manifest written, but is not tagged. Then if* $\mathrm{eff}_{\sigma_1}(p) \neq \mathsf{none}$*, $p \in \Pi(\sigma_j)$ for every j, and Theorem 55 fails.*

Proof. p gains no tag in any run, so by Lemma 40 $W^\circ_{\sigma_j}(p) = W^\circ_{\sigma_1}(p)$ for all j, and $\mathrm{dir}_{\sigma_j}(p) = \mathrm{dir}_{\sigma_1}(p) \neq \mathsf{none}$ if the bump was direct (and, by Theorem 47, $\mathrm{prp}_{\sigma_j}(p) \geq \mathrm{prp}_{\sigma_1}(p)$ if it was propagated, since p never releases at h in the sense Lemma 40 requires). Either way the plan's first disjunct holds at every σ_j, so $\mu_j \geq 1$ for all j and no run empties the plan. □

This is why SPEC §13.10a tags whatever is released, including private, internal and artefact-less packages, and why the convergence check of SPEC §19.6 (`E199`) admits exactly one exclusion, held packages. A permanent exception to convergence would not be a caveat to document; it would disable the one check that detects the orphan failure of SPEC §13.7a, and disable it precisely for the internal-only packages where nobody is watching a registry.

5.8 G7: channel discharge

Theorem 57 (**G7**, channel discharge)**.** *Suppose d is released in run k on channel ch, so that* $\mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$ *for $j > k$. Then for every unit u and every $j > k$,* $\mathrm{pcf}_{\sigma_j}(u, d) \neq ch$*: the channel d has arrived on is never proposed for it again.*

Proof. Write $cur = \mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$; this is read from d's tag, not from anything computed in the run. Every branch of Definition 23 ends in the test "if $target = cur$, return $\perp$". A result of ch would require $target = ch = cur$, which that test rejects. The transition branch fails one step earlier where it applies: having arrived at t, d no longer matches an $f \neq t$, and for $f = *$ with $t = \mathsf{stable}$ the match requires $cur \neq \mathsf{stable}$, which fails. □

Corollary 58 (common ledger discharge)**.** *After any release, $W^\circ()$ excludes every delivered directive. The no-op and transition checks of Theorem 57 independently prevent re-proposing the channel already recorded in the baseline. These are complementary checks; the baseline comparison is not a substitute for fresh-work admission.*

Both axes therefore discharge through the common undelivered window, with the channel axis adding its no-op check. Under the retry invariant both survive a partial failure unchanged.

This also marks the boundary of what Theorem 57 gives. It says the channel d arrived on is not re-proposed; it does *not* say nothing is proposed. The historical CCME 1.0.0 counterexample in proof obligation 79 used its unsplit window to let a second, older directive propose a different channel on every run. CCME 2.0.0's fresh-window gate discharges that directive after release.

5.9 G8: axis independence

Theorem 59 (**G8**, axis independence)**.** *The channel assignment* $\mathrm{chan}_\sigma(\cdot)$ *is a function of* σ*, h and the configuration that does not read any quantity computed by phase 3. Consequently, altering the bump directives* $\mathrm{pb}(u), \mathrm{pd}(u), \mathrm{ps}(u)$ *of any set of units leaves* $\mathrm{chan}_\sigma(\cdot)$ *unchanged, and in particular a* `W208` *suppression removes no channel assignment and causes no package to lose a channel change it would otherwise have had.*

Proof. By Definition 24, phase 1 reads the units' channel directives, $\mathrm{src}_\sigma(u)$, the traversal, $W^\circ_\sigma(\cdot)$, $\mathrm{canc}(\cdot,\cdot)$, and, through Definition 23, $\mathrm{base}_\sigma(\cdot)$. By Definition 25, phase 2 reads phase 1's output and the same fixed quantities. None of these is a phase-3 output: $\mathrm{prp}_\sigma(\cdot)$, $\mathrm{eff}_\sigma(\cdot)$ and the admission predicate appear nowhere in Definitions 23 to 25. So $\mathrm{chan}_\sigma(\cdot)$ is determined before phase 3 runs and is invariant under any change confined to phase-3 inputs. □

The dependency between the axes is thus a single directed edge, from channels to bumps, and Theorem 59 is the statement that there is no edge back. Two operational consequences follow, both worth stating because they are what a plan reader needs: a `W208` in a plan means a caret did not reach, not that a channel directive failed; and a package may legitimately appear in a plan with a channel change and no bump (`W202`), or with a bump and no channel change, and neither shape indicates a defect.

Remark 60 (the converse does not hold). CCME 1.0.0 stated **G8** as "suppressing a bump under SPEC §9.3a never suppresses a channel, *and vice versa*". CCME 2.0.0 states only the one-way result above. The old proof observed that the passes share only $\mathrm{chan}_\sigma(\cdot)$, which phase 3 reads and never writes. That argument establishes exactly the direction proved above. The converse is false, and intentionally so: a channel suppressed by `W200` leaves $\mathrm{chan}_\sigma(d)$ at its baseline value, and a bump from sources on another channel is then suppressed by Item (A6). Proof obligation 82 states the correction.

6 Determinism, idempotency, resumption

SPEC §17.2 states three obligations that are usually treated as informal quality properties. They are provable in this model, and two of them are corollaries of Section 5.

Theorem 61 (determinism)**.** $\Pi(\sigma)$*, the versions* $\nu_\sigma(\cdot)$*, the planned publish order and the ordered planning diagnostic list are functions of* $(\sigma, h, \mathrm{config})$ *alone. Two conforming implementations, on two machines, at two times, produce identical plan output. Concurrent task completion, logs, and other operational events are outside this claim.*

Proof. Every quantity in Sections 2 and 3 is defined by a closed expression in σ, h and the configuration: Definitions 9 and 14 from ancestry and tags; Definition 12 from commit messages and the configured types, separator and limits; Definition 19 from the graph at h; Definitions 23 to 26 and 28 from those; Definitions 15 and 33 from those. None mentions wall-clock time, commit dates, tag creation order, or the run history.

Three points admit a choice and each is resolved by a stated total order. Conflicting propagated channels and disagreeing origin channels resolve to the byte-wise least package name

(W160, Definition 23). Competing directives resolve by SPEC §11.6 precedence: newest commit first, then last unit within the commit, a total order on units, since commits carry a topological index and units an index within their commit. Publish-order ties resolve by byte-wise least name (Proposition 34). SPEC §17.2 additionally requires every collection reaching the output to be sorted by package name, then commit topological index, then unit index, so no unordered container's iteration order is observable. A function of fixed inputs with all ties broken by total orders is single-valued. □

Corollary 62 (idempotency)**.** *Running the engine twice with no intervening commits and no publish yields the same plan. Under the retry invariant, running it after a fully successful publish yields the empty plan.*

Proof. The first is Theorem 61: the state is unchanged, so the plan is. The second is Theorem 51 applied to each $p \in \Pi(\sigma_1)$, which is Theorem 55 with n reached in one run; the invariant excludes both input drift and newly visible catch-up. □

Theorem 63 (resumption after partial failure)**.** *Assume the retry invariant. Let* $\sigma_2 = \mathrm{run}(\sigma_1, F)$ *with induced blocked set* B*. Then*

$$\Pi(\sigma_2) \;=\; F \cup B, \qquad \textit{and} \qquad \nu_{\sigma_2}(p) = \nu_{\sigma_1}(p) \;\; \textit{for every } p \in F \cup B.$$

Proof. $\subseteq$: $\Pi(\sigma_2) \subseteq \Pi(\sigma_1)$ by the first paragraph of the proof of Theorem 55, and every $p \in \Pi(\sigma_1) \setminus (F \cup B)$ was published, hence excluded by Theorem 51. $\supseteq$ and the version equality: each $p \in F \cup B$ was planned in run 1 and not released, so Theorem 48 applies verbatim. □

Theorem 63 is the exact statement of what the operator is promised after a failed run under the retry invariant: the next run does the rest of the same job, at the same numbers without coordinator state to clean up. An ambiguous external publish outcome still requires SPEC §19.4 reconciliation. It is also what SPEC §19.6 checks. After a run in which nothing failed or was blocked, $F \cup B = \emptyset$, so the re-planned set must be empty; a non-empty one is E199. Inside the retry invariant it contradicts Theorem 63; outside it, SPEC §19.6 reports the changed admission inputs for renewed review.

Remark 64 (what determinism forbids). Theorem 61 is the reason SPEC §13.11 may permit optimisation of the computation but not of its result. An implementation may cache identical traversals, invert loops, cache lexical parses and publish concurrently; it may not introduce parallelism whose scheduling affects the iteration order of any collection that reaches the output. Section 8 records the precise conditions under which such transformations preserve those observables.

7 The staleness dual

SPEC §9.2 walks *down* from the packages a commit changed. An audit command wants the opposite walk: given a package, which of its dependencies is it behind, and by how much? SPEC §13.7b gives that walk and asserts that the two agree, calling disagreement an implementation bug. This section proves the agreement under the normative CCME 2.0.0 eligibility checks.

```
staleSources(D):
    W     = freshWindow(D)                          # the *target's* window
    edges = graph.restrictedTo(config.propagation.kinds)
```

```
dist  = shortestPathsUp(D, edges)   # P -> edge count of shortest P -> ... -> D
out   = {}
for u in unitsIn(W):
    if bumpOf(u) == none:                                continue
    b = (u.propagate == 'inherit') ? bumpOf(u) : u.propagate
    if b == none or u.depth == 0:                        continue
    sources = sourcePackages(u)
    if sources is empty:                                 continue
    if D in sources:                                     continue
    reaching = { P in sources : P in dist }
    if reaching is empty:                                continue
    level = min({ dist[P] for P in reaching })
    if level > u.depth:                                  continue
    if D not in resolve(u.propagateScope):               continue
    if cancelledFor(commitOf(u), D):                     continue
    if not resolvableBy(sources, D, channel):             continue
    for P in reaching:
        out |= { (P, u, level, b) }
return out
```

Definition 65. $\mathrm{stale}_\sigma(d) = \{(p, u, \ell, b)\}$ is the set the procedure returns, with $\ell = \delta(\mathrm{src}_\sigma(u), d)$ and $b = \mathrm{pb}(u)$.

Theorem 66 (duality)**.** *For every state σ and package d,*

$$\mathrm{stale}_\sigma(d) \neq \emptyset \iff \mathrm{prp}_\sigma(d) \neq \mathsf{none}, \qquad \textit{and} \qquad \bigsqcup\{\, b : (p, u, \ell, b) \in \mathrm{stale}_\sigma(d) \,\} = \mathrm{prp}_\sigma(d).$$

Proof. It suffices to show that u contributes a row to $\mathrm{stale}_\sigma(d)$ exactly when $\mathrm{adm}_\sigma(u, d)$, since the contributed rows all carry $b = \mathrm{pb}(u)$ and $\mathrm{prp}_\sigma(d)$ is the join of $\mathrm{pb}(u)$ over admitted units (Definition 29).

The guards map one for one. `bumpOf(u) == none`, `b == none` and `u.depth == 0` are Item (A1). `sources is empty` and the `level > u.depth` test are Item (A2): by Lemma 20 the traversal admits d iff $\delta(\mathrm{src}_\sigma(u), d) \leq \mathrm{pd}(u)$ and $d \notin \mathrm{src}_\sigma(u)$, and `level` is exactly that distance, because `shortestPathsUp` gives $\delta(\{p\}, d)$ for each p and the minimum over $p \in$ `reaching` is $\delta(\mathrm{src}_\sigma(u), d)$ (a shortest chain from a set is the shortest among the chains from its members). `if D in sources: continue` is the $d \notin \mathrm{src}_\sigma(u)$ of Definition 19, read from the other end: it is the same clause as seeding `seen` with the sources. `D not in resolve(u.propagateScope)` is Item (A3); `unitsIn(W)` with W the target's window is Item (A4); `cancelledFor` is Item (A5); `resolvableBy` is Item (A6). So the guarded set is the admitted set.

Finally `out` is non-empty iff at least one unit passes, and each such unit has $\mathrm{pb}(u) \neq \mathsf{none}$, so non-emptiness is equivalent to $\mathrm{prp}_\sigma(d) \neq \mathsf{none}$. □

Example 67 (the audit, on the failed run)**.** At σ_1 of Example 30, where `core` has published and `cli` has not, the audit run for $d =$ `cli` walks up from `cli`, finds `core` at `dist`[`core`] $= 1$, and returns the single row

$$\mathrm{stale}_{\sigma_1}(\texttt{cli}) = \{\, (\texttt{core},\ u,\ 1,\ \mathsf{patch}) \,\},$$

whose join is $\mathsf{patch} = \mathrm{prp}_{\sigma_1}(\texttt{cli})$, as Theorem 66 requires. Run the same audit for `ui`, which did publish, and it is empty: $C \notin W^\circ_{\sigma_1}(\texttt{ui})$ removes the only candidate. This is the question an operator actually asks after a failed run (which of my packages are behind, and behind what), answered without recomputing a plan.

The three clauses the proof leans on are exactly the three SPEC §13.7b warns an implementation drifts on, and it is worth naming what each drift produces. Excluding d from its own unit's sources: without it, a package that both changes and consumes a sibling in one unit audits as behind itself. Measuring `level` from the whole source set rather than per ancestor: taking `dist`$[p]$ for each p separately over-reports the level and so under-reports staleness, exactly in the diamond cases. And resolving `inherit` to the unit's own bump before testing it against `none`.

Proposition 68 (the tag-level screen is necessary, not sufficient)**.** *The screen of* SPEC *§13.7b is only a candidate test. If* $\mathrm{adm}_\sigma(u, d)$ *for some unit* u *with a source* p *that has released the work in* c_u*, then* $\mathrm{basec}_\sigma(p)$ *is not an ancestor-or-self of* $\mathrm{basec}_\sigma(d)$*. The converse fails.*

Proof. Suppose $\mathrm{basec}_\sigma(p) \preceq \mathrm{basec}_\sigma(d)$. Since p released the work, $c_u \preceq \mathrm{basec}_\sigma(p)$, so $c_u \preceq \mathrm{basec}_\sigma(d)$ by transitivity, hence $c_u \in \mathrm{reach}(\mathrm{basec}_\sigma(d))$ and, by Definition 9, $c_u \notin W^\circ_\sigma(d)$, contradicting Item (A4).

For the converse, a provider may be legitimately ahead of a consumer with nothing owed: the units between them may all declare `^none` or depth `+0`, may be excluded by `Propagate-Scope`, may reach d only beyond their declared depth, or may travel edges of a kind not in `propagation.kinds`. Each gives a state where the screen fires and no bump is admitted. □

The screen is therefore a way to *find* candidates cheaply, by comparing two tags rather than walking a window, and never a way to decide releases; using it to decide would manufacture bumps that no commit asked for. There is also a case it cannot see at all: p and d released at the same commit, in the same run, with d published first. Ancestry cannot distinguish them and no rule over tags can. That case is prevented rather than detected, by publishing in dependency order (Definition 33).

Remark 69 (historical source of the eligibility guard)*.* CCME 1.0.0 omitted this guard, so wherever SPEC §9.3a suppressed a bump the two formulations disagreed. Proof obligation 81 preserves the counterexample; CCME 2.0.0 makes the repair normative.

8 Cost

CCME 3.0.1 makes SPEC §13.11 normative for semantic equivalence and informative for cost and technique. An implementation must produce the plan SPEC §13 defines, but need not attain any bound below. The accounting assumes decoded, indexed inputs; comparison, allocation, adapter, matching and emitted output costs remain. This section states safe transformations and the parser bound SPEC §18.3 relies on; it makes no general optimality claim.

Notation: P packages, E edges, N commits and M parent edges reachable from h, C commits in the union windows, U units, B input bytes, T tags, R scope and changed-path resolution work, $I = \sum_u |\mathrm{scope}(u)|$ resolved unit-to-package incidences, and Z contribution/provenance incidences. Per unit, D is the number of targets it reaches, S its source-set size and Σ its resolved scope-set size.

Phase	Literal	Achievable	Proved in
Train/fresh windows (SPEC §13.3)	$O(P(N+M))$	$O((k_s+k_f+2)(N+M))$	Proposition 70
Candidate traversals (SPEC §13.7)	$O(U(P+E))$	input-dependent $+O(Z)$	Theorem 45, Proposition 72
Per-target predicates (SPEC §9.3a)	$O(D(S+\Sigma))$	$O(D+S+\Sigma)$	Proposition 75
Channel resolution (SPEC §13.8)	$O(P \cdot U)$	$O(U+I+P)$	Proposition 76
Publish order (SPEC §19.2)	historical $O(P+E)$	$O(P \log P + E)$	Proposition 34
Parse/resolve (SPEC §13.4, SPEC §20.7)	—	$O(B+R+I)$; lexing linear	Proposition 77

The candidate-traversal row is not a bound on the complete planner. A cached walk can safely be reused when its source set, depth and edge kinds are identical; it must retain each unit's own distance, fresh-window membership, cancellation, channel eligibility, self-source exclusion, and provenance. Even outside the merged source set, a nearby ineligible unit cannot lend its short distance to a farther eligible unit. Per-unit predicate checks and output construction add work; this report does not prove that all such work fits the bucket traversal bound. The proposition below proves a geometric candidate-set relation, not full semantic equivalence of an unannotated merged traversal.

The publish-order row reads differently from the others: the historical CCME 3.0.0 bound $O(P+E)$ holds for an arbitrary topological order, while CCME 3.0.1 accounts for the bytewise-least ready package that SPEC §19.2 requires at every step. Selecting it is an extract-min over the ready set, so the achievable cost is $O(P \log P + E)$ with a binary heap (each package enters and leaves the heap once, each edge is relaxed once), and Proposition 34 proves that this order is well defined and machine independent, not that it is linear.

8.1 Windows

Proposition 70 (window bucketing)**.** *$\{W_\sigma(p) : p \in \mathcal{P}\}$ has at most k_s+1 distinct members, where k_s is the number of distinct defined values of $\mathrm{scom}_\sigma(\cdot)$. Computing $\mathrm{reach}(\cdot)$ once per distinct baseline commit and testing membership by lookup costs $O((k_s+1)(N+M))$ reachability work and, after restricting membership to the C candidate commits, $O((k_s+1) \cdot C)$ bits.*

Proof. By Definition 9, $W_\sigma(p)$ depends on p only through $\mathrm{scom}_\sigma(p)$; packages sharing a baseline commit share a window, and packages with no stable baseline share $\mathrm{reach}(h)$. So the map from the k_s baseline commits (plus the undefined case) onto distinct windows is surjective, giving the count. The costs follow: k_s+1 traversals of at most N commits and M parent edges, followed by C membership bits per distinct window. □

The fresh window is determined by the pair $(\mathrm{scom}_\sigma(p), \mathrm{basec}_\sigma(p))$ and likewise has at most k_f+1 classes. Both k_s and k_f can be as large as P, and the added one accounts for packages without a baseline. Commit-generation numbers can reject candidates and bound a walk but cannot prove ancestry across branches. A positive result still needs a reachability query or a precomputed index. These $(N+M)$-sensitive bounds correct the historical 3.0.0 C-only row; 3.0.1 accounts for reachable history. C bounds stored candidate membership, not necessarily the history visited to establish ancestry.

8.2 Safe traversal reuse

The superseded bucketing proposal grouped units by window class and directive tuple and traversed from the union of their sources. That key is insufficient: unit identity affects admission, cancellation, scope, inherited channel, self-source exclusion and output provenance. CCME 3.0.1 removes the proposal and identifies identical graph-walk inputs as one safe memoisation key. Other optimisations still require an equivalence argument. The following historical result characterises why the naive merge is unsafe.

Definition 71. For $S \subseteq \mathcal{P}$ let $\delta^+(S, d)$ be the least length of a chain of *at least one* edge from a member of S to d, and $R(S, \ell)^+ = \{d : \delta^+(S, d) \leq \ell\}$. This is the naive positive-edge candidate set; CCME 2.0.0 additionally requires attribution to an eligible unit and that unit's self-source exclusion.

Proposition 72 (limit of naive merged bucketing)**.** *Let $u_1, \ldots, u_m$ share a depth ℓ and let $S_i = \mathrm{src}_\sigma(u_i)$, $S = \bigcup_i S_i$. Then*

$$\bigcup_i R(S_i, \ell) \;\subseteq\; R(S, \ell)^+,$$

the two sides agree on every $d \notin S$, and they differ exactly on the packages

$$X \;=\; \{\, d \in S \;:\; \delta^+(S, d) \leq \ell \;\; \textit{and} \;\; \forall i,\; d \in S_i \;\textit{or}\; \delta(S_i, d) > \ell \,\},$$

which $R(S, \ell)^+$ admits and no individual traversal does. In particular, if no $\mathrm{src}_\sigma(u_i)$ contains a package reachable from that same set within ℓ edges, then $X = \emptyset$ and bucketing is exact.

Proof. Inclusion: if $d \in R(S_i, \ell)$ then $d \notin S_i$ and $\delta(S_i, d) \leq \ell$, so the witnessing chain has length at least 1 and starts in S, giving $\delta^+(S, d) \leq \ell$.

Agreement off S: let $d \notin S$ and $\delta^+(S, d) \leq \ell$. The witnessing chain starts at some $p \in S_i$, and $d \notin S_i$, so $d \in R(S_i, \ell)$.

The difference: for $d \in S$, membership of $R(S, \ell)^+$ says only that some chain of length $1..\ell$ reaches d from S, whereas membership of $\bigcup_i R(S_i, \ell)$ requires some i with $d \notin S_i$ and $\delta(S_i, d) \leq \ell$. Negating the latter gives the stated X. □

Example 73 (the bucket that over-admits)**.** On Figure 1, take the single unit

$$u \;=\; \texttt{fix(core,ui)\^{}major:}\;\; \texttt{x},$$

so $S_1 = \{\texttt{core}, \texttt{ui}\}$ and $\ell = 1$. Per Example 21, $R(S_1, 1)$ excludes `ui`, which takes the unit's own `patch` directly. But $\delta^+(S_1, \texttt{ui}) = 1$ by the edge `ui` → `core`, so `ui` $\in R(S_1, 1)^+$: the merged rule gives `ui` the propagated `major` as well, publishing it two levels higher than the specification says, with no diagnostic to show for it. Here $X = \{\texttt{ui}\}$.

Remark 74 (what X costs, and what SPEC §13.11 says about it). The warning box in SPEC §13.11 is about the opposite error, subtracting the merged source set S, which withholds the bumps of a package that is a source of one unit and a legitimate target of another, and now requires a per-unit self-source test. Merely using $R(S, \ell)^+$ over-admits on X: a unit whose own scope-set contains both a package and a dependent of it within the depth bound. Then the dependent, which SPEC §9.2 excludes from that unit's targets because the unit bumps it directly, receives the unit's *propagated* bump as well, a difference whenever `Propagate` names a value above the unit's own bump, as in `fix(core,cli)^major`. Proof obligation 84 preserves the 1.0.0 counterexample that motivated the current rule.

8.3 Hoisting

Proposition 75 (the resolvability predicate is loop-invariant)**.** $\mathrm{rsv}_\sigma(u, d)$ *as computed by the source scan*

$$\exists p \in \mathrm{src}_\sigma(u) : \ \mathrm{chan}_\sigma(p) = \mathsf{stable} \ \vee \ \mathrm{chan}_\sigma(p) = \mathrm{chan}_\sigma(d)$$

is equal, for every d, to the membership test of Definition 26 against the precomputed set $\mathrm{sch}_\sigma(u)$*. Evaluating it once per unit and testing membership per target costs $O(D + S)$ where the literal form costs $O(D \cdot S)$; hoisting c_u and* $\mathrm{ps}(u)$ *likewise takes the unit from $O(D(S + \Sigma))$ to $O(D + S + \Sigma)$.*

Proof. A set contains $\mathrm{chan}_\sigma(d)$ exactly when some $p \in \mathrm{src}_\sigma(u)$ has $\mathrm{chan}_\sigma(p) = \mathrm{chan}_\sigma(d)$, and contains stable exactly when some p has $\mathrm{chan}_\sigma(p) = \mathsf{stable}$; the disjunction of the two membership tests is therefore the disjunction of the two existentials, which is the source scan. The set has one entry per *distinct* channel among the sources. The predicate reads d only in that one comparison, and Item (A3) and c_u do not read d at all, so all three may be computed before the target loop. □

The cost matters because of where it lands: a unit carrying `^^` across a wide scope-set admits thousands of targets from a source set of one or two packages, and the literal reading re-walks those sources thousands of times to re-derive an answer that never changed. Hoisting is not an optimisation to be justified by profiling; it is what the predicate's own dependency structure already says.

8.4 Inverted channel resolution

Proposition 76 (inversion)**.** *Building the candidate lists by pushing from units to the packages their scope-sets resolve to yields, for each p, the same list as scanning the window for directives naming p, provided units are pushed in* SPEC *§11.6 precedence order. It costs $O(U + I + P)$ against the literal $O(P \cdot U)$; the incidence term cannot be omitted because those appends are output work.*

Proof. A package p appears in the pushed list of u exactly when u carries a `Channel` directive, $p \in \mathrm{scope}(u)$, and $c_u \in W^\circ_\sigma(p)$, which is the pulled form's membership condition read in the other direction, so the two lists have the same elements. They have the same order because the push visits units in the order the pull would sort them into. The push performs one pass over units, each contributing $|\mathrm{scope}(u)|$ appends against scope resolutions SPEC §13.4 has already paid for, and one flat pass over packages. □

Two obligations come with the inversion and neither is optional. Units must be pushed in SPEC §11.6 order, or the first-match-wins loop below silently picks a different directive, a determinism bug that surfaces only when one package is named by two commits. And the `W186` count is over the whole candidate list, not the prefix examined before the winner was found, so the list must be complete before it is read; an implementation that fuses the two passes and evaluates a package as soon as its first candidate arrives loses `W186`.

8.5 Parsing

Lexical parsing and resolution have distinct cache keys. Lexing is pure in the repository identity, immutable record identity, parser/schema identity and parsing configuration. Scope and changed-path resolution additionally depends on the workspace package/path index, graph

and resolution configuration. A lexical cache hit therefore does not cache R or I, and record immutability does not imply a whole-plan cache. Vectors 82h–82j exercise direct-channel resolution, inherited-channel provenance and per-unit admission; they do not exhaustively validate these cache keys.

Proposition 77 (linear parsing, SPEC §20.7)**.** *A conforming parser reads a message of n bytes in $O(n)$ time and $O(1)$ auxiliary space beyond the parsed result, with no input causing superlinear behaviour.*

Proof sketch. SPEC §20 specifies every step as a forward index scan over the byte array with no backtracking: separator detection compares a fixed token at line starts; header parsing advances a cursor through type, scope-set, inline directives, `!` and `:` , each transition determined by the current byte and a bounded amount of already-scanned state; footer detection tests each line against a fixed set of key shapes. Each byte is examined a bounded number of times, and the working state is a fixed number of indices and flags. No construct requires re-examining a prefix, which is the property that distinguishes it from a backtracking regular-expression engine and is why SPEC §18.3 can promise bounded work on untrusted commit messages. □

This is the one bound in the table stated for the parser rather than the planner, and it is here because SPEC §18.1's threat model includes commit messages authored by anyone who can open a pull request. A superlinear parser would make that a denial-of-service surface in CI; Proposition 77 is why the specification prescribes an index scan and relegates regular expressions to an appendix.

9 Proof obligations

Formalising CCME 1.0.0 at `713f1a0b` exposed six failure classes. Each obligation below states the claim, a minimal counterexample, the consequences, a repair, and a conformance vector. Three concern SPEC §13.7c, one SPEC §13.7b, one SPEC §9.3, and one the optimisation advice of SPEC §13.11.

None is a defect in the reference implementation. In two cases (proof obligations 79 and 81), the implementation already uses a mechanism absent from the specification. A second implementation following only the written contract would therefore be incorrect. Source references below are relative to `services/dispat/internal/plan/` and cite the line numbers of the released `services/dispat/v1.7.2` tag.

Released CCME 2.0.0 resolves them as follows: proof obligation 79 by W_{fresh}; proof obligations 80 and 83 by the full retry invariant and fixed-state wording; proof obligation 81 by applying channel eligibility in the upward audit; proof obligation 82 by stating axis independence in the bump-to-channel direction only; and proof obligation 84 by retaining each unit's own source exclusion when bucketed. The counterexamples remain regression evidence for those normative conditions. Except where a paragraph explicitly names CCME 2.0.0 or a current theorem, every numbered SPEC § reference and quoted guarantee in this section refers to the CCME 1.0.0 snapshot at `713f1a0b`; present tense describes the historical rule being executed, not the released 2.0.0 text.

9.1 Current obligation: the graduation summary contradicts its specific rule

Proof obligation 78 (CCME 2.0.0 ğ13.9 does not preserve ğ11.5's train target)**.** Section 11.5 explicitly defines graduation as apply(S, E) using the same aggregate E over the train window as ğ11.4. That rule supplies the intended stable core. Section 13.6, however, defines the direct bump from fresh work and phase 3 likewise produces fresh propagation; ğ13.9's generic stable branch then applies their `effective(P)` to the stable baseline. On a channel-only graduation, fresh effective can be `none` although an older minor or major change established the train. Read literally, the summary loses the aggregate that ğ11.5 requires. The model follows the more specific ğ11.5 rule; the specification should make ğ13.9 delegate to it or carry a separately named train target.

A second wording inconsistency occurs inside ğ11.5: E185 is said to fire when the target is not greater than the "baseline core", although normal graduation in its own table removes the prerelease suffix while retaining the same core. The meaningful monotonicity comparison is full SemVer precedence against the prerelease baseline, under which the stable version is greater. This is a specification-text obligation, not evidence of a runtime failure. Current Theorem 48 proves only that fixed version inputs produce the same result on retry; it does not resolve contradictory formulas by itself. A conformance vector should enter a train on an aggregate minor or major target, then perform a channel-only graduation and require that target's stable version without E185.

9.2 Historical 1.0.0 obligations

9.3 The prerelease window never discharges

Proof obligation 79 (the 1.0.0 G4/G6 claims fail on a prerelease train)**.** Under the 1.0.0 single-window rule, ledger discharge and convergence fail after a successful prerelease publish. Current Theorems 51 and 55 use W_{fresh} and do not make that claim.

Counterexample (bump axis). This is SPEC vector 95b, run twice. Baselines `core@1.5.0-beta.0` and `cli@2.0.1-beta.0`, both with an older stable baseline; `cli` depends on `core`; one commit `fix(core)^: x`.

Run 1 plans `core@1.5.0-beta.1` and `cli@2.0.1-beta.1` and publishes both. Run 2, at the same h: by Lemma 40 neither window changed, because Definition 9 measures from the last *stable* tag and no stable tag was written. Under the 1.0.0 direct and propagated rules, `core` and `cli` each receive `patch` again, both packages are planned again, and SPEC §11.4 increments the counters to `beta.2`. Run 3 gives `beta.3`. The plan never empties, and `E199`, the convergence check of SPEC §19.6, which admits no exclusion but held packages, fires after the first fully successful run of any prerelease train.

Counterexample (channel axis). Theorem 57 says the channel a package arrived on is not proposed for it again. It does not say nothing is proposed. Let p have stable baseline `1.2.0` and a window containing an older `release(p)%rc` and a newer `release(p)%beta`. Run 1: SPEC §11.6 precedence gives the newer directive, p is released `1.2.1-beta.0` (`W186` noting the conflict), and the operator's stated intent is honoured. Run 2: the window is unchanged, so both directives are still candidates; `%beta` now proposes nothing (`W199`, Theorem 57) and the *older* `%rc`, which SPEC §11.6 had ruled had lost, wins by default and p is released `1.2.1-rc.0`. Run 3: `%rc` is now inert and `%beta` proposes `beta` again, computing `1.2.1-beta.0`,

which does not exceed the baseline `1.2.1-rc.0`, so the run stops with `E195`. A repository that did nothing wrong is wedged, having published a version onto a channel its newest directive did not ask for.

What broke in 1.0.0. Its G4 (no double release), G6 (convergence), and G7 channel-settling claim (the channel axis settles) all fail, and with them SPEC §17.2's idempotency clause and SPEC §19.6's convergence check, for every package on a prerelease train, which is to say for the entire feature the prerelease flow exists to support.

The repair. The window answers one question, what has p not yet released, and SPEC §13.3 correctly measures it from the stable tag because SPEC §11.4 recomputes a train's target over the whole train. The missing distinction is that the same set is then used to answer a second question: is anything still *owed*? Separate them. Define the *fresh window*

$$W^{\circ}_{\sigma}(p) \;=\; W_{\sigma}(p) \setminus \mathrm{reach}(\mathrm{basec}_{\sigma}(p)),$$

which for a package whose baseline is stable equals $W_{\sigma}(p)$, and for one on a train excludes the commits its own prereleases already published. Then:

(R1) Definition 14 discharges on $W^{\circ}_{\sigma}(p)$, so a package whose prerelease shipped the work counts as a source (which SPEC §13.4a already intends: the artefact is public);

(R2) plan membership (Definition 15) requires a contribution from a commit in $W^{\circ}_{\sigma}(p)$, while the version computation (SPEC §11.4) keeps reading $W_{\sigma}(p)$, so the train's target still accumulates over the whole train;

(R3) candidate collection in Definitions 24 and 25 is restricted to $W^{\circ}_{\sigma}(p)$, which is what stops the superseded directive of the second counterexample from voting again.

With (R1)–(R3), Lemma 40 strengthens to $W^{\circ}_{\sigma'}(p) = \emptyset$ for *every* released p, prerelease or not, because $\mathrm{basec}_{\sigma'}(p) = h$. Fresh discharge repairs the prerelease counterexample. Current Theorems 51, 53 and 55 separately state the full retry-invariant conditions needed for persistence, non-widening, and bounded convergence; (R1)–(R3) alone do not imply them.

Evidence. The reference implementation has exactly this predicate, under a different name and undocumented in SPEC: `containedInBaseline` (`plan.go`, lines 1836–1855) reports whether a commit has already been published by the package's baseline tag, and its callers implement (R1) in `sourcePackages`, (R2) by gating `NewWork` on it, and (R3) in both `resolveChannels` and `propagateChannels`. Its comment states the consequence the specification does not: without it the engine "would re-warn ...on every run until graduation".

Regression vectors. Vector 133 is the released 2.0.0 form of the bump-axis case. Historically, (i) vector 95b, run twice: the second run's plan must be empty. (ii) The two-directive window above, run three times: run 2 must be empty, and `E195` must not arise.

9.4 Window-scoped suppression lapses, so the admitted set can grow

Proof obligation 80 (the 1.0.0 G5 claim lacks suppression stability)**.** The 1.0.0 proof assumed "same sources, same depth, same graph at h", although window-scoped suppression can change the source set. CCME 2.0.0 makes the needed stability part of the full retry invariant used by current Theorem 53.

The root cause is a design decision taken three times, in three sections, each time for a good local reason: suppression is scoped to the window. A `cancel` stops counting once the package releases (SPEC §13.4a); a hold does the same (SPEC §13.4a); and a correction "reaches only undischarged work", so a `Deletes` is a no-op for a package that has released its target (SPEC §7.4.2, Remark 13). Each rule is right on its own terms. Together they mean that the set of units a package acts as a propagation source for is *monotone in the state*, which is precisely what **G5** says it is not.

Counterexample (cancellation). `cli` depends on `core`. Three commits: A carrying `feat(core)^: streaming reader`, then B carrying `cancel(core)`, then C carrying `fix(core): unrelated`, in that ancestry.

Run 1: $\mathrm{canc}(A, \texttt{core})$ holds and $A \in W_{\sigma_1}(\texttt{core})$, so `core` is not in $\mathrm{src}_{\sigma_1}(u_A)$ and u_A propagates to nothing; `core` releases `1.4.3` from C alone (SPEC vector 41). $\Pi(\sigma_1) = \{\texttt{core}\}$: `cli` is not planned.

Run 2: `core` is tagged at h, so $W_{\sigma_2}(\texttt{core}) = \emptyset$, so $\mathrm{disch}_{\sigma_2}(\texttt{core}, A)$ holds, so `core` re-enters $\mathrm{src}_{\sigma_2}(u_A)$ by the discharge disjunct of Definition 14. Now u_A propagates: $A \in W_{\sigma_2}(\texttt{cli})$, `cli` is not cancelled, and the bump is admitted. $\Pi(\sigma_2) = \{\texttt{cli}\} \not\subseteq \Pi(\sigma_1)$.

Counterexample (corrections). The same shape without any `cancel`. Let B instead carry `chore(core): drop it` with `Deletes: <A>`. In run 1 the correction discards A's record for `core`, so u_A has no source and reaches nothing; `core` still releases, on C. In run 2, $W_{\sigma_2}(\texttt{core})$ is empty, so by SPEC §7.4.2 the correction is a discharged no-op for `core` (`W209`), A's record stands again, and u_A propagates to `cli` exactly as before. This route is worth stating separately because `W209` reads as reassurance (the correction did not need to do anything) when it is in fact the moment the suppressed propagation returns.

What broke in 1.0.0. Its unrestricted G5 claim and the "nothing is added" step of its G6 proof failed. Theorem 55's conclusion survives here (`cli` releases in run 2 and run 3 is empty) but the variant argument does not, and the security reading of **G5** does: whether `cli` is released at all depends on whether `core`'s publish succeeded. Inducing a publish failure changes the set of packages a commit eventually releases, which is the inference SPEC §18.1 relies on **G5** to forbid.

The repair, and why the obvious one is wrong. The tempting fix is to make cancellation sticky, never letting a cancelled package become a source, by defining $\mathrm{disch}_\sigma(p, c) \wedge \neg\mathrm{canc}(c, p)$. It is wrong: it breaks SPEC vector 72, where `cancel(core)` lands *after* `core` published the work and the consumers' catch-up must still proceed. Ancestry cannot separate the two cases, since in both the cancel is in the history at h and in both `core` carries a tag at h. The difference is only whether the released version counted the unit, and that is not recoverable from the state.

The recommendation is therefore to restate the guarantee rather than change the algorithm: **G5** holds for every unit none of whose sources is suppressed at h, which is hypothesis (H2), and the widening in the remaining case is intended behaviour, since neither a `cancel` nor a `Deletes` touches code (SPEC §10.1, SPEC §7.4.2) and the work of A is in fact inside the published `core@1.4.3`. What the specification should not do is claim a subset property that the discharge rule of SPEC §13.4a deliberately violates, and SPEC §18.1 should not rest a security argument on the unrestricted form.

Evidence. The cancellation counterexample was run against the reference implementation, which follows SPEC §13.4a literally and reproduces it: with the history above and `core` tagged at h in the second state, the first plan releases `core` alone and the second releases the consumer as well, reporting it as a *catch-up from core*. The engine is doing what the specification says; it is **G5** that does not describe it.

Regression vector. Vector 134 is the released 2.0.0 form. In the historical pair, each history above is run twice: run 2 plans `cli`, the plan is documented as a superset of run 1's, and the `W193` naming `core` as the origin is present so that the widening is visible to whoever reads the plan rather than silent.

9.5 The staleness dual omits the resolvability test

Proof obligation 81 (SPEC §13.7b's two formulations disagree)**.** SPEC §13.7b states that `staleSources(D)` "is non-empty exactly when §9.2 assigns `D` a non-`none` propagated bump", and that disagreement is an implementation bug. The procedure tests scope, depth, window, and cancellation, but has no counterpart to Item (A6).

Counterexample. SPEC vector 95 (also edge case 60m): `feat(core)^%beta: x` where `core` moves onto `beta` and its consumers stay on `stable`. SPEC §9.3a suppresses every consumer's bump with `W208`, so $\mathrm{prp}_\sigma(\texttt{ui}) = \texttt{none}$, and SPEC is explicit that "`feat(core)^%beta` releases `core` alone". But `staleSources(ui)` passes every guard it has and returns the row $(\texttt{core}, u, 1, \texttt{patch})$. The audit reports `ui` as behind a dependency for which no release is owed, and by SPEC §13.7b's own words the implementation is now buggy by definition.

The repair. Add the hoisted predicate of Proposition 75 to the procedure, immediately after the `cancelledFor` test:

```
        srcChan = { channel[P] : P in sources }        # ğ9.3a, as in ğ9.2
        if not ('stable' in srcChan or channel[D] in srcChan):  continue
```

With it, Theorem 66 holds unconditionally and its hypothesis can be dropped.

Evidence. The reference implementation cannot exhibit the disagreement, because `Plan.StaleSources` (`plan.go`, line 1217) returns the rows recorded by the downward traversal rather than re-deriving them from an independent upward walk. The duality is therefore true there by construction and untested; an implementation that follows SPEC §13.7b literally, as a second implementation would, disagrees on every prerelease workspace.

Regression vector. Vector 135 is the released 2.0.0 form; historically, vector 95 was extended: assert that `staleSources(ui)` is empty, and that the randomised down-versus-up cross-check SPEC §13.7b recommends is run on a workspace containing at least one prerelease train.

9.6 Axis independence is one-directional

Proof obligation 82 (the "vice versa" in **G8**)**.** SPEC §13.7c states **G8** as "suppressing a bump under SPEC §9.3a never suppresses a channel, and vice versa", and proves it by observing that

the two passes share only $\text{chan}_\sigma(\cdot)$, which phase 3 reads and never writes. That argument proves one direction only, and the other is false.

Counterexample. Let `core` and its consumer `app` both sit on `stable`, and let one commit carry `feat(core)^%beta++1`: the caret propagates a bump to `app` and the `++1` propagates the `beta` channel to it, which is exactly the pairing SPEC §9.3a prescribes for taking consumers along. Suppose `app`'s own window also contains `release(app)%rc`. By Definition 25 a direct directive beats every propagated one, so $\text{chan}_\sigma(\texttt{app}) = \texttt{rc}$ and the propagated `beta` is discarded. Phase 3 then computes $\text{sch}_\sigma(u) = \{\texttt{beta}\}$, finds $\text{chan}_\sigma(\texttt{app}) = \texttt{rc} \notin \text{sch}_\sigma(u)$ and no source on `stable`, and suppresses the bump with `W208`. Had the propagated channel been applied, `app` would have been on `beta` and the bump admitted. The suppression of a channel proposal caused the suppression of a bump.

The repair. State **G8** as Theorem 59 does: the channel assignment is computed by phases 1–2 and reads no phase-3 output, so no bump suppression can affect it. The dependency in the other direction is real, is the entire content of SPEC §9.3a, and should be documented as such rather than denied. The operational consequences SPEC §13.7c draws from **G8** are unaffected: they only ever use the direction that holds.

Regression vector. Vector 136 is the released 2.0.0 form. The historical workspace above asserts that `app` is released on `rc` with no propagated bump and a `W208`, and that the `W208` is a consequence of the channel precedence rule rather than of anything the caret did.

9.7 Inherited channels read a baseline that release moves

Proof obligation 83 (SPEC §9.3's replay-independence claim)**.** SPEC §9.3 states of the `inherit` resolution that "the result is deterministic, does not depend on which earlier run published the origin". The second half is false: resolution rule 2 reads $\text{ch}(\text{base}_\sigma(q))$, and the origin's own release moves it.

Counterexample. `app` depends on `core`; `core` sits at baseline `1.5.0-beta.0` over stable baseline `1.4.2`; `app` at `2.0.0`. Two commits: C_1 carrying `feat(core)^++1: x` (no explicit channel value, so the propagated channel is `inherit`) and C_2 carrying `release(core)%stable: graduate`.

Run 1: `core`'s own channel is `stable` (the direct directive), but the inherited value for `app` is $\text{ch}(\text{base}_{\sigma_1}(\texttt{core})) = \texttt{beta}$, so `app` is planned at `2.0.1-beta.0` while its provider graduates to `1.5.0`. Suppose `core` publishes and `app` fails. Run 2: $\text{ch}(\text{base}_{\sigma_2}(\texttt{core})) = \texttt{stable}$, the proposal is inert (`W199`), and `app` is planned at `2.0.1`. Both runs plan `app`, at different versions, with every completed release stable: the 1.0.0 cross-run version-stability claim fails. CCME 2.0.0 instead fixes inherited baseline-channel state as one component of its full retry invariant, as required by current Theorem 48. Run against the reference implementation, the two plans are exactly as computed here (`app`: `beta` → `stable`, `2.0.1-beta.0` → `2.0.1`).

What breaks, and what does not. Determinism (Theorem 61) is unaffected: each plan is still a pure function of its state. What fails is the *cross-run* half of the specification's sentence, and with it channel stability for unreleased dependents of a graduating origin. Note also that run 1's own output is operationally odd: a consumer is dragged onto a train in the same run that ends it. Both effects have one cause, phase 1 reading baselines so that it cannot depend

on phase 2's output (SPEC §9.2); the price of that layering is that `inherit` answers with the origin's pre-graduation channel.

The repair. Scope the specification's sentence to a fixed state (determinism per run, which is what Theorem 61 proves), and state the drift explicitly. Resolving `inherit` against the origin's *resolved* channel instead would remove the drift but requires a fourth phase, which SPEC §9.2's no-circularity argument was written to avoid; the fresh window of proof obligation 79 does not remove it either, since the commit remains pending for the failed dependent. The honest fix is documentation plus the vector below.

Regression vector. Vector 137 is the released 2.0.0 form: the two commits above, two runs: run 2 must plan `app` at `2.0.1` on `stable`, and the plan should carry the `W199` that records why the run-1 proposal lapsed.

9.8 Bucketed traversal over-admits a unit's own sources

Proof obligation 84 (the 1.0.0 bucketing remedy has its own edge)**.** CCME 1.0.0 SPEC §13.11 warned that subtracting the merged source set from a bucketed traversal silently withholds bumps, and prescribes admitting "any node reached across at least one edge within the depth bound". By Proposition 72 that prescription over-admits the set X: a package that is itself a source of the unit that reaches it.

Counterexample. One unit, `fix(core,cli)^major: x`, where `cli` depends on `core`. SPEC §9.2 seeds `seen` with both sources, so `cli` is not a propagation target and takes the unit's own `patch`. Under the merged rule, `cli` is reached from `core` across one edge inside the depth bound and takes the propagated `major` as well: a two-level difference in the published version, with no diagnostic.

The repair. Bucket the traversal, then re-decide the few targets that are themselves sources. Since the bucket key fixes the propagated bump, the only question for such a d is whether *some* unit in the bucket admits it, that is whether $\exists i : d \notin S_i \wedge \delta(S_i, d) \leq \ell$. The candidates are confined to S, which is small, and the per-unit source sets are already materialised, so the check is cheap. Equivalently, and more simply, exclude from bucketing any unit whose own source set contains a package reachable from that set within the depth bound.

Evidence. The reference implementation does not bucket: it traverses once per unit (`propagate.go`, line 401), so it is unaffected. The obligation is on the advice, which a performance-minded second implementation would follow.

Regression vector. Vector 138 is the released 2.0.0 form. Historically, SPEC Appendix B.7a, whose vectors test exactly this class of almost-equivalence, gains one: `fix(core,cli)^major` must publish `cli` as a patch, bucketed or not.

10 Conformance

Each result below is testable, and most are already tested. The table maps every theorem to the specification section it formalises, the conformance vectors of SPEC Appendix B that

exercise it, and the test in the reference implementation (`services/dispat/internal/plan/`) that stands behind it. The point of the mapping is that a result with no vector and no test is a claim resting on the proof alone, and the last column is where that shows.

Result	SPEC	Vectors	Test
Theorem 45 (**G1**)	13.7c	67, 68	`TestPropagationDepthIsShortestPath`
Theorem 47 (**G2**)	13.7c	66, 74, 76	`TestCatchUpAfterConsumerFailure`, `TestCatchUpForNeverReleasedConsumer`
Theorem 48 (**G3**)	13.7c	70, 84, 137	`TestCatchUpOnATrainWithPublishedOwnWork`
Theorem 51 (**G4**)	13.7c	66, 83, 133	`TestCatchUpDischargesExactlyOnce`
Theorem 53 (**G5**)	13.7c	67, 68, 134	`TestCatchUpNeverWidensBlastRadius`
Theorem 55 (**G6**)	13.7c, 19.6	77, 77a, 133	`TestPrereleaseTrainConvergesAfterRelease`, `TestPropagatedTrainWorkConverges`
Proposition 56	13.10a	77, 77a	`TestFixedConvergesWhenNothingPending`
Theorem 57 (**G7**)	13.7c, 9.3	60r, 97c	`TestChannelAxisConverges`, `TestTransitionIsIdempotent`
Theorem 59 (**G8**)	13.7c, 9.2	94, 95, 97, 136	`TestPropagatedStableNeverGraduates`, `TestCaretAloneDoesNotDragConsumersOntoATrain`
Theorem 61	17.2	82	`TestConflictingDirectChannelsPickTheNewest`, `TestConflictingPropagatedChannelsPickTheNewest`
Corollary 62	17.2	82, 83	`TestNoCatchUpWhenConsumerIsFresh`
Theorem 63	17.2, 19.3	70, 84	`TestCatchUpAfterConsumerFailure`
Theorem 66	13.7b	135	`TestStaleSourcesAgreesWithPropagation`
Proposition 68	13.7b	—	—
Proposition 34	19.2	69, 80a	— (executor, outside `plan`)
Proposition 70	13.11	—	`TestPlanningQueriesGitOncePerPackage`
Proposition 72	13.11	82c–82j, 138	— (no traversal reuse implemented)
Proposition 75	9.3a, 13.11	82c, 82d	`TestCaretAloneDoesNotDragConsumersOntoATrain`
Proposition 76	13.8, 13.11	82e, 82f	`TestDirectChannelBeatsPropagated`
Proposition 77	20.7	99	`FuzzParse` (`pkg/ccme`)

Three rows deserve comment.

Theorem 66 has a test, and the test cannot fail. As proof obligation 81 records, `Plan.StaleSources` returns the rows the downward traversal recorded, so `TestStaleSourcesAgreesWithPropagation` compares a quantity with itself. That is a reasonable engineering choice (an independent upward walk would be a second implementation of the same rule, with its own bugs) but it means the duality is asserted rather than checked here, and it is the reason proof obligation 81 went unnoticed. The randomised cross-check SPEC §13.7b recommends is the thing that would catch it, and it needs a workspace with a prerelease train to do so.

Theorem 55 is the row whose test predates the normative CCME 2.0.0 fresh-window rule. `TestPrereleaseTrainConvergesAfterRelease` opens with the words "the reported bug": a package that entered a beta train and published `1.0.0-beta.0` re-appeared in every subsequent plan as `beta.1`, `beta.2`, and so on. That is exactly proof obligation 79, found in operation rather than on paper, and fixed in the engine. CCME 2.0.0 now specifies W_{fresh}, so a second implementation following the current text discharges the prerelease work as the engine does.

Proposition 72 has vectors and no test because the reference implementation does not perform the optimisation the vectors are about. Its traversal is per unit (`propagate.go`), which is the

literal reading and is trivially correct. The vectors of Appendix B.7a exist for implementations that do optimise, and proof obligation 84 adds one more.

Randomised differential validation

After this report was written, its model was validated differentially against the reference implementation, most recently in the historical campaign with the released `v1.7.2` binary (the `v1.4.0`, `v1.6.0`, `v1.7.0` and `v1.7.2` campaigns agree per-seed on all 3,000 instances, so the figures below hold for those releases): a ∼300-line oracle implementing the report model as it stood against the 1.0.0 snapshot, a seeded generator of instances (5–20 packages, layered graphs, 4–12 units, mixed tag states), and a realiser turning each instance into a real repository for the binary to plan.

On 1,500 instances inside the historical premises then called (H1) and (H2), across two disjoint seed ranges, the two plans agreed *exactly*: package sets, versions, bumps and channels. On 1,000 instances over the full directive grammar the 1,097 divergent package-rows classified *completely* into four predicted classes and none outside them: 240 prerelease-train discharges and 58 suppression-lapse and downstream-cascade variants are proof obligations 79 and 80 observed at scale; 409 channel-entry patches are SPEC §11.4's deliberate one-patch rule (`W204`), which neither this report's model nor the companion article's transcribes; and 390 bump-axis liveness cases arise because the oracle implements the companion article's compact admission, which drops this report's clause (A1), a transcription gap of the article, caught by the differential run exactly as intended. The obligations plus these two recorded omissions form a complete divergence taxonomy for the grammar exercised, which is the strongest empirical statement this historical campaign makes about its scope. These records do not validate changed CCME 2.0.0 admission.

The fresh 7 September 2026 E1 campaign used the released `dispat 1.8.2 (linux_arm64)` binary on 3,000 generated instances. The runner validates Git fixtures and CLI exits. All 1,500 core instances agreed with no invalid sample. The full profile intentionally generates regressive prereleases: 997 plans were accepted, of which 546 matched the historical oracle, while 503 complete diagnostic plans were semantically refused with `E195`. Four refused dossiers had no row difference, so 550 comparisons were diagnostic-clean. All proposed regression package sets in the 503 dossiers were independently checked, including held proposals. The full profile had 1,914 divergent package rows, assigned by the campaign's heuristic classifier to PG1 862, PG2 375, PG2x 14, and PG3? 663, with zero unclassified rows. The canonical manifest binds all four shards to extractor SHA-256 `3d5eef3e...11e52`; its paths and hashes identify the four records used for these totals. Because the oracle implements the historical model and the labels—especially PG3?—are heuristic, these counts are taxonomy evidence, not CCME 2.0.0 conformance.

E1 uses tmpfs and pre-run `git fsck`; the revised exit checker separates validated E195 refusals from execution failures. The first stopped attempt and the 1.8.1 records remain preserved. Seed 580's earlier invalid fixture remains unresolved; its fresh success is not a product-fix claim, since planner source is unchanged and the fixture environment changed.

A separate check in the 7 September 2026 campaign ran vectors 133–138 against the released Dispat 1.8.2 binary; all six passed. The retained raw before/after plan streams for 134 and 137 confirm package counts, versions and catch-up behaviour. Vector 135 exercises the observable downward eligibility result; it is not an independent implementation of the upward audit, so the proof remains the support for full duality. This is selected conformance evidence, not an exhaustive proof of the specification.

The official image is `yohimik/dispat-alpine@sha256:4c0ad4b8...e7bfd`. The provenance

bundle fixes peeled release tag commit `a7011cbcb9e8` and verifies the official binary against its release asset. All 753 files in the final checksum manifest and all eight completion/identity markers verified. The same bundle records the other fresh checks without converting them into proof claims. E2 repeated orphan recovery twice: no refused tag, the same recovered version, then an empty third plan. E3 reproduced the established historical replay profile over five releases per repository (36/42 exact for `google-cloud-node`, 7/64 for `conventional-changelog`); it does not claim those releases followed CCME. E4's two recovery repetitions converged in 24/24 episodes within four runs (31 and 32 total runs) with no structural G3/G5 errors. Its eight fresh timing cells span 100–10,000 packages: medians range from 0.017 s for the 100-package layered case to 19.354 s for the 10,000-package chain, with 15–118 MiB observed maximum RSS. The recorded fixtures passed `git fsck`; the pad generator was used only after a small fixture matched the ordinary generator's commit IDs, refs, messages, and log output. Measurements used a contended Apple-silicon Docker host capped at two CPUs and 4 GiB, so they are observations rather than clean comparisons. E5 freshly reran its orphan, clean concurrent-push and conflicting concurrent-push Dispat cells; their 8, 12, and 14 checks passed. Its nine competitor cells remain historical evidence.

A Notation

Symbol	Meaning	Defined
$\mathcal{K}$, h	commits; the fixed HEAD	Definition 1
$\mathrm{reach}(c)$, $c' \preceq c$	ancestors-or-self of c; c' is one of them	Definition 1
$\mathcal{P}$, E, $\mathrm{dep}(p)$	packages; dependency edges; dependents of p	Definition 3
$\delta(S, d)$	least edge count from a member of S to d	Definition 5
$R(S, \ell)$	the traversal: $\{d \notin S : \delta(S, d) \leq \ell\}$	Definition 19
$\mathbb{B}$, $\sqcup$	$\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}$; their join	Definition 6
$\mathcal{V}$, $\mathcal{C}h$, stable	versions; channels; the stable channel	Definition 7
$\mathrm{ch}(v)$, $\mathrm{core}(v)$, $\mathrm{cnt}(v)$	channel, core triple, prerelease counter of v	Definition 7
$\mathrm{apply}(v, b)$	v with bump b applied	Definition 7
σ, $\sigma \leq \sigma'$	engine state (the reachable tags); inclusion	Definition 8
$\mathrm{base}_\sigma(p)$, $\mathrm{basec}_\sigma(p)$	baseline version of p; its commit	Definition 8
$\mathrm{sbase}_\sigma(p)$, $\mathrm{scom}_\sigma(p)$	stable baseline of p; its commit	Definition 8
$W_\sigma(p)$	train window of p	Definition 9
$W^\circ_\sigma(p)$	undelivered (fresh-admission) window	Definition 9
$\mathcal{U}$, c_u, $\mathrm{scope}(u)$	units; a unit's commit; its resolved scope-set	Definition 12
$\mathrm{bump}(u)$	the bump u's type and `!` map to	Definition 12
$\mathrm{pb}(u)$, $\mathrm{pd}(u)$, $\mathrm{ps}(u)$	bump-axis directives of u	Definition 12
$\mathrm{pc}(u)$, $\mathrm{pcd}(u)$, $\mathrm{pcs}(u)$	channel-axis directives of u	Definition 12
$\mathrm{canc}(c, p)$, $\mathrm{held}(p)$	the two suppressors	Definition 14
$\mathrm{disch}_\sigma(p, c)$	p has already released the work in c	Definition 14
$\mathrm{src}_\sigma(u)$	source packages of u	Definition 14
$\mathrm{sch}_\sigma(u)$	channels of u's sources	Definition 26
$\mathrm{rsv}_\sigma(u, d)$	d can resolve what u's sources release	Definition 26
$\mathrm{adm}_\sigma(u, d)$	u propagates to d	Definition 28
$\mathrm{chan}_\sigma(p)$	the channel p is released on	Definition 25
$\mathrm{dir}_\sigma(p)$, $\mathrm{prp}_\sigma(p)$, $\mathrm{eff}_\sigma(p)$	direct, propagated, effective bump	Definition 29
$\Pi(\sigma)$, $\nu_\sigma(p)$	the plan; p's planned version	Definition 15
$\mathrm{run}(\sigma, F)$	one run, failing on F	Definition 17
$\mathrm{stale}_\sigma(d)$	the upward audit of SPEC §13.7b	Definition 65
(H1), (H2)	standing hypotheses	Section 5

Diagnostic codes (`E200`, `W208`, and the rest) are those of SPEC §16 and are used here with their registry meanings.

References

[1] *Conventional Commits: Monorepo Extension (CCME) 2.0.0.* `released CCME 2.0.0`, `github.com/yohimik/dispat`. The historical audit uses the `CCME 1.0.0 snapshot`. GPL-3.0-or-later. Cited throughout as SPEC.

[2] *Conventional Commits: Monorepo Extension (CCME) 3.0.1.* `released CCME 3.0.1`, `github.com/yohimik/dispat`. Corrected algorithm guidance; not an implementation or experimental subject of this report.

[3] *dispat*: a release engine for polyglot monorepositories. `github.com/yohimik/dispat`. The reference implementation; the release computation discussed here is `services/dispat/internal/plan`.

[4] *Conventional Commits 1.0.0.* `conventionalcommits.org`. CCME is a strict superset.

[5] T. Preston-Werner. *Semantic Versioning 2.0.0.* `semver.org`. Supplies the version precedence of Definition 7.

[6] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels.* RFC 2119, IETF, 1997. The requirement levels SPEC is written in.

[7] H. Garcia-Molina and K. Salem. *Sagas.* In Proceedings of the ACM SIGMOD International Conference on Management of Data, 1987. The pattern the publish protocol of Section 4 implements, with git tags as the log.

[8] A. B. Kahn. *Topological sorting of large networks.* Communications of the ACM 5(11), 1962. The publish order of Proposition 34.

[9] S. Chacon and B. Straub. *Pro Git*, 2nd edition. Apress, 2014. The commit DAG, reachability and tags of Definitions 1 and 8.

[10] Semyon F. (yohimik). *HOL Guard pull request 2813: opt-in Dispat release-start review.* Open, unmerged pull request, revision `132bb5d105e1234b9f81fddf69267ec488e2990a`, 6 September 2026.